



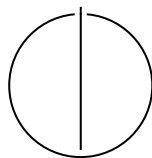
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

**Optimizing Model Distribution in
Containerized Distributed Machine
Learning Systems**

Bohdan Garchu





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

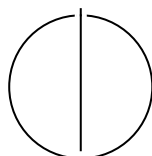
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

**Optimizing Model Distribution in
Containerized Distributed Machine
Learning Systems**

**Optimierung der Modellverteilung in
containerisierten Systemen für verteiltes
maschinelles Lernen**

Author:	Bohdan Garchu
Examiner:	Prof. Dr.-Ing. Jörg Ott
Supervisor:	M.Sc. Giovanni Bartolomeo, M.Sc. Navidreza Asadi
Submission Date:	17.08.2026



I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 17.08.2026

Bohdan Garchu

Acknowledgments

First and foremost, I would like to express my deepest gratitude to my supervisors, Giovanni Bartolomeo and Navidreza Asadi, for their support throughout this project. They gave me the freedom to work independently and at my own pace, while always providing valuable advice and guidance. Without their support, this work would not have been possible.

I would also like to thank the Chair of Connected Mobility at the Technical University of Munich and Prof. Dr.-Ing. Jörg Ott for giving me the opportunity to write my thesis here.

In addition, I am deeply grateful to my family for their continuous support, especially during the more challenging moments along the way. I am equally grateful to my friends and my girlfriend for their encouragement, patience, and for always being there for me.

With this thesis, an important chapter of my life comes to an end, while a new one begins. I am grateful for everything I have learned and for everyone who has been part of this journey.

Abstract

Containers have become the standard unit for packaging and deploying cloud-native applications. Their layered image format, however, is poorly suited to workloads that combine stable code with large, frequently changing artifacts, such as machine learning models. A changed artifact can require substantial parts of an image to be rebuilt and redistributed, making frequent updates costly.

Production systems often address this problem by distributing large artifacts separately through object storage or MLOps pipelines. This requires additional infrastructure and separates artifact versioning from the application image. Existing container-native approaches address individual aspects of the problem: lazy-loading systems such as eStargz fetch image data on demand, while 2DFS builds image partitions independently. Neither combines independent build with lazy loading nor supports updating artifacts in a running container.

We present 2DFS-STAR, an OCI-compatible build, distribution, and runtime system that combines independent image partitioning, lazy loading, and in-place refresh. 2DFS-STAR keeps large artifacts within the image while fetching their contents on demand and prefetching selected partitions. When a new image version is published, 2DFS-STAR can refresh a running container without restarting the application, transferring only data that changed.

We evaluate 2DFS-STAR using language-model and multi-model video analytics workloads across cloud and resource-constrained edge environments. 2DFS-STAR builds images faster than Docker and eStargz while remaining close to 2DFS, with pull and cold-start latencies comparable to eStargz. In-place refresh reduces update latency by 8.3–85.9× in the cloud and 7.0–265.6× at the edge, while transferring orders of magnitude less data than container replacement.

Contents

Acknowledgments	iii
Abstract	iv
1 Introduction	1
1.1 Problem Statement	1
1.2 Approach and Contributions	3
1.3 Thesis Outline	4
2 Background	5
2.1 The OCI Image Model	5
2.2 Building Images	6
2.3 Running Images: containerd and Snapshotters	7
2.4 Lazy Image Loading with eStargz	7
2.5 Two-Dimensional Filesystems	9
2.6 Updating Running Containers and Applications	10
2.7 Model Distribution in Distributed Machine Learning	11
2.8 Capability Comparison	11
3 System Design	13
3.1 Functional Requirements	13
3.1.1 Build	13
3.1.2 Distribution	13
3.1.3 Update and Refresh	14
3.2 Non-functional Requirements	14
3.2.1 Performance	14
3.2.2 Resource Efficiency	15
3.3 System Architecture	15
3.3.1 Combining Partitioning with Lazy Loading	16
3.3.2 Image Format	17
3.3.3 2DFS-STAR Builder	18
3.3.4 2DFS-STAR Registry	18
3.3.5 Client-Side Stack	19

3.3.6	Subscription Mode	21
3.3.7	Default Fallback Path	22
4	Implementation Details	23
4.1	2DFS-STAR Builder	23
4.2	2DFS-STAR Registry	24
4.3	Client-Side Stack	24
4.3.1	Snapshotter Configuration	24
4.3.2	Delta Refresh	26
4.3.3	Integration	28
4.3.4	Subscription Mode	29
5	Evaluation	30
5.1	Experimental Setup	30
5.2	Build Time	33
5.3	Image Update Time	34
5.4	Pull Time and Cold-Start Latency	35
5.5	Live Refresh	35
5.6	Resource Usage	36
5.6.1	Build	37
5.6.2	Pull	38
5.7	Video Analytics Application	38
5.8	Performance at the Edge	39
5.9	Compression Method	40
5.10	FUSE Passthrough	41
5.11	Prefetch	42
6	Conclusion	44
6.1	Summary of Findings	44
6.2	Applicability Beyond Model Serving	44
6.3	Limitations	44
6.4	Future Work	45
6.4.1	Lifting the Refresh Preconditions	45
6.4.2	Application-Aware Coordination	46
6.4.3	Complementarity with Checkpointing	46
	Abbreviations	47
	List of Figures	48

Contents

List of Tables	50
Bibliography	51

1 Introduction

Containers have reshaped how applications are developed, deployed, and managed. A container packages an application and its dependencies as a portable unit. The same unit runs consistently across development machines, cloud platforms, and edge devices. Containers also improve resource utilization and simplify scaling. These properties have made them the default deployment format for server workloads [8, 11, 13, 61].

Machine learning (ML) workloads expose both the strengths and the limits of this format. A model-serving runtime often depends on complex frameworks and exact software versions, all of which a container can package into a reproducible, isolated deployment unit. This is particularly valuable for computationally intensive ML serving across distributed systems [2, 62, 63, 66]. However, the same image also couples the serving runtime to large model artifacts that may follow a very different update cycle.

Model weights may be retrained or replaced while the serving code remains unchanged. Conversely, the serving code may be patched while the weights remain valid. A single image keeps these parts versioned and prevents dependency drift, but it also forces each model update through the full cycle of rebuilding, redistributing, and replacing the container.

In production, serving code often remains stable across many model revisions, while training and fine-tuning produce new weights regularly [21, 35, 60, 64]. Deployments of the same application may also require different subsets of the packaged models [38, 39, 46, 52]. The image remains the unit of build, transfer, and update in both cases. It therefore turns independent choices about code and models into one all-or-nothing operation.

1.1 Problem Statement

This coarse granularity originates in the image format and carries into the tools built on top of it. Open Container Initiative (OCI) images, including Docker images, combine read-only filesystem differences through a union filesystem. Each build instruction that changes the filesystem, such as `RUN`, `COPY`, or `ADD`, produces an immutable layer. That layer records the changes relative to the layer below it. Whole-layer sharing enables caching and reuse across images. However, layer immutability also couples otherwise independent changes. Modifying one file invalidates its layer and every layer above it.

Deleting or replacing a file does not remove its bytes from an earlier immutable layer. These hidden bytes remain part of the image and can contribute to image bloat [42, 66].

2DFS [2–4] removes part of this coupling. It extends the OCI layer model with a two-dimensional filesystem layer. The layer stores independent image components as separately addressable allotments. A change to one allotment leaves the others untouched. This design enables parallel builds, fine-grained caching, and on-demand partitioning of a large image into smaller images.

However, 2DFS retains the existing runtime behavior. Developers define the partitioning semantics, but clients still fetch every selected allotment eagerly. A new image version also requires a new container. Consequently, 2DFS improves how an image is built and stored without changing how a runtime loads or updates it.

Pull latency therefore remains part of the deployment cost. The image pull often dominates the cold start after an image has been built and distributed [21, 35, 37, 50]. A conventional layer is a compressed tar archive without random access. The runtime must download and decompress the entire layer before it can use any file inside. This sequence creates a delay between workload placement and process startup. Allocated compute remains idle throughout that delay, especially for data-intensive ML applications.

Lazy-loading systems reduce this delay by fetching image contents on demand [8, 15, 26, 27]. This thesis builds on eStargz [15], which makes files inside a compressed layer seekable. The runtime can fetch only the chunks that an application accesses. This approach can greatly reduce startup latency when a workload reads only part of an image.

A conventional eStargz pipeline starts with an existing OCI image and rewrites each layer into the seekable format after the build. This additional conversion pass adds build overhead. After conversion, the runtime can fetch individual files on demand, but the image remains organized as conventional one-dimensional layers. eStargz therefore does not provide the independent build and partitioning offered by 2DFS.

The image lifecycle also continues after the first deployment. Teams retrain models, swap drivers, and revise configurations while the surrounding runtime remains valid. Immutable layers offer no way to expose these changes selectively inside a running container. Even a small update therefore requires a new image and a replacement container. This process increases update latency, consumes bandwidth and storage, and interrupts service.

ML systems incur this cost frequently because model artifacts can change every few minutes to every few hours [5, 21, 31, 35, 51, 52, 60, 64]. Rolling [7] and blue-green [53] deployments reduce service disruption by starting a second container and shifting traffic to it [52, 53, 64]. The replacement still needs the new image, which must be built, stored, and transferred. It also begins without the warm runtime state of the previous

instance. Large images and slow edge links make this cold start especially costly [7]. A live refresh avoids these costs when the process can continue running and only its mounted data changes.

Other domains face the same problem. Simulation workloads [1, 6, 19, 22, 23], cloud-native 5G core and radio-access components [32, 33, 43, 58], game engines [49], and web applications [16] all package large static assets with the code that uses them. These assets can change independently of the surrounding code. A small asset update still incurs the transfer and restart cost of replacing the whole container.

These observations reveal four gaps that define the scope of this thesis.

1. Lazy loading lowers pull latency but increases build cost. The conversion pass rewrites every layer after an already sequential build. Pull-time savings therefore come with additional build-time overhead.
2. Conventional container build caches operate at layer-chain granularity. A change invalidates the affected layer and all dependent layers above it. The builder therefore cannot rebuild only the changed image component.
3. 2DFS supports selective builds but pulls complete allotments. A client must fetch each requested allotment in full and cannot load individual files lazily.
4. A new image version cannot reach a running container. Current tooling requires a restart and a full image pull to expose the new version.

1.2 Approach and Contributions

We present 2DFS-STAR, an OCI-compatible pipeline that combines the independent build and partitioning of 2DFS with lazy image loading and in-place refresh. This combination addresses build, pull, and update costs while preserving registry-based distribution.

This thesis makes the following contributions.

- An OCI-compatible 2DFS-STAR build and distribution pipeline that produces lazy-loadable 2DFS allotments in one pass. Its build overhead remains close to plain 2DFS.
- Prefetch annotations derived from the 2DFS pull tag. A client can fetch a selected partition eagerly while loading the rest on demand. The client can also disable prefetch and load the whole image lazily.

- An in-place refresh for running containers that reuses unchanged cached content and exposes updated image contents without restarting the application.
- A subscription mode that monitors a registry tag and automatically refreshes a running container when a new image version is published.
- A benchmark suite and an evaluation of language-model and multi-model video analytics workloads in cloud and edge environments. The evaluation measures build time, pull latency, refresh latency, and resource usage against Docker, eStargz, and 2DFS. 2DFS-STAR builds faster than Docker and eStargz while remaining close to 2DFS. Its pull performance is comparable to eStargz. Live refresh reduces update latency by 8.3 to 85.9 times in the cloud and 7.0 to 265.6 times at the edge. An update to one of the three evaluated language models transfers 605 B to 16.3 KB. Replacing the container transfers 6.4 to 58.3 GB.

1.3 Thesis Outline

Chapter 2 introduces the OCI image format, the container build pipeline, lazy loading with eStargz, and baseline 2DFS design. It also positions 2DFS-STAR against related work. Chapter 3 defines the requirements and presents the 2DFS-STAR architecture across 2DFS-STAR builder, 2DFS-STAR registry, and the client-side stargz snapshotter stack. Chapter 4 describes implementation details for each component. Chapter 5 presents the benchmark methodology and results. Chapter 6 summarizes the findings, discusses the design’s limitations, and identifies directions for future work.

2 Background

2DFS-STAR builds on several layers of the container stack. OCI defines the image format, and image builders produce its contents. At runtime, containerd and its snapshotters mount the image as a filesystem. eStargz extends this stack with lazy loading, while 2DFS adds independent image partitioning. This chapter introduces each part in that order. It then examines approaches to runtime updates and how this stack affects model building, transfer, and updates in distributed ML. A final capability comparison positions 2DFS-STAR against its direct baselines.

2.1 The OCI Image Model

An OCI image is a content-addressed bundle containing everything needed to run a container [42]. The image forms a directed acyclic graph with four types of objects: an *index*, one or more *manifests*, a *config* for each manifest, and the filesystem *layers*. The index sits at the root. Each manifest describes an image for one platform and references its config and ordered list of layers.

The registry stores every object as a blob and identifies it by a hash of its contents. Identical objects therefore share one stored blob across images. This content-addressed model provides the basis for registry caching and deduplication.

A layer represents a set of filesystem changes. The first layer modifies an empty root filesystem. Each subsequent layer adds, replaces, or removes files relative to the layers below it. A runtime combines the ordered layers through a union filesystem such as OverlayFS to construct the container's root filesystem.

On disk, a layer is a gzip-compressed tar archive with two distinct identifiers. The *digest* hashes the compressed blob and identifies the layer in the manifest and registry. The *diffID* hashes the uncompressed layer. The image config lists the diffIDs in layer order to describe the root filesystem.

Content addressing also makes the image graph rigid. A change to one layer produces a new digest. The manifest that references the layer then changes, followed by the index that references the manifest. A single layer edit therefore propagates to the root of the image graph.

OCI descriptors provide the format's extension mechanism. A descriptor identifies a blob by its digest, media type, and size. It can also carry a map of string *annotations*.

```
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.oci.image.manifest.v1+json",
  "config": {
    "mediaType": "application/vnd.oci.image.config.v1+json",
    "digest": "sha256:2a1f...",
    "size": 1234
  },
  "layers": [
    {
      "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
      "digest": "sha256:7c9e...",
      "size": 5678,
      "annotations": {
        "containerd.io/snapshot/stargz/toc.digest": "sha256:4b6d..."
      }
    }
  ]
}
```

Listing 2.1: An abbreviated OCI image manifest with one annotated layer descriptor.

Annotations leave the referenced blob unchanged. A registry that does not recognize an annotation can still store it, and a runtime can safely ignore it. eStargz uses annotations to identify seekable layers and expose metadata required for lazy loading (Section 2.4).

Listing 2.1 shows an abbreviated OCI manifest. The config and each layer appear as descriptors, while annotations attach metadata to an individual layer.

2.2 Building Images

An image builder converts a build descriptor into an OCI image through a front end and a back end. Docker pairs the `dockerfile` front end with the BuildKit back end. The front end translates a Dockerfile into a Low-Level Build (LLB) dependency graph, which BuildKit evaluates and executes. Each instruction that changes the filesystem, such as `RUN`, `COPY`, or `ADD`, produces an immutable layer. The layer records the changes relative to the state beneath it.

BuildKit can execute independent graph nodes in parallel, but filesystem-dependent layer generation remains sequential. The order of filesystem operations affects the

result, and a later instruction may depend on the exact state left by an earlier instruction. The builder captures each state by mounting the working filesystem in a build container and extracting its changes. Extraction cost grows with the size of the copied files. A small edit to a large file therefore costs as much as copying the complete file.

The ordered pipeline also couples cache invalidation across layers. A layer records changes against the state below it. Modifying that state changes the input to every later instruction and invalidates the layers produced from it.

2.3 Running Images: containerd and Snapshotters

containerd pulls images and manages their runtime lifecycle. It represents each mounted layer as a *snapshot* and delegates snapshot creation to a pluggable *snapshotter*. A snapshotter creates and mounts filesystem snapshots for image layers and container writable state. Completed image-layer snapshots can be retained for reuse and removed when no longer needed. Command-line clients such as `ctr` and `nerdctl` communicate with containerd through its socket.

containerd's default `overlayfs` snapshotter stores unpacked image layers locally [14]. It must therefore download and unpack a complete layer before making its contents available. A *remote snapshotter* removes this requirement. It mounts a layer from its remote store and presents that mount as a snapshot. The runtime can then start a container before all layer contents are present locally [15]. containerd communicates with a remote snapshotter through a proxy-plugin daemon over a Unix socket. The daemon implements the same snapshotter interface as built-in plugins.

Filesystem in Userspace (FUSE) allows a userspace process to implement a filesystem that is accessed through the kernel's standard filesystem interface [36]. The snapshotter uses FUSE to serve remote image layers. It makes file metadata available locally and fetches file contents from the registry when an application first reads them.

2.4 Lazy Image Loading with eStargz

A remote snapshotter needs random access to fetch individual files from a layer. A plain gzip tar layer does not provide it. The runtime cannot extract a file without decompressing the preceding archive contents. The layer also contains no per-file digest, which prevents independent verification of data smaller than the complete layer [15].

eStargz adds random access and per-file verification while remaining a valid gzip layer. An eStargz image can therefore pass through an ordinary registry and run on a

```
{
  "name": "bin/busybox",
  "type": "reg",
  "size": 833104,
  "offset": 126,
  "digest": "sha256:8b7c559b...",
  "chunkDigest": "sha256:8b7c559b..."
}
```

Listing 2.2: A TOC entry for one regular file in `stargz.index.json`.

runtime that does not understand the extension. Such a runtime treats the layer as a conventional gzip tar archive.

eStargz inherits its seekable layout from stargz, the tar.gz format introduced by Google’s CRFS project [26]. A plain tar.gz archive compresses its contents as one continuous stream. Stargz compresses each regular file independently. It first divides a large file into *chunks* and compresses each chunk separately. The format concatenates the resulting streams into one valid gzip file. Each stream can be decompressed without the surrounding bytes. A byte offset can therefore locate one file or chunk inside the layer. The inner compression can use gzip or a chunked Zstandard variant. Chunked Zstandard keeps the same seekability while offering better performance [12].

A table of contents (TOC) records the byte offsets. The TOC is a JavaScript Object Notation (JSON) file named `stargz.index.json`, stored as the final tar entry. It records each file’s name, type, uncompressed size, byte offset, and content digest. It records the same information for each chunk of a large file. In gzip-based eStargz, a 51-byte empty gzip member forms the footer. Its extra field encodes the byte offset of the TOC.

The runtime first reads the footer and follows its offset to the TOC. This metadata is enough to mount the complete filesystem tree without fetching file contents. When an application reads a file, the runtime obtains its offset and size from the TOC. It then issues a Hypertext Transfer Protocol (HTTP) Range Request for that part of the layer. The OCI distribution specification already supports these requests. Listing 2.2 shows one TOC entry.

The same metadata establishes trust in lazily fetched contents. A layer descriptor carries the mandatory `containerd.io/snapshot/stargz/toc.digest` annotation. Its value is the TOC digest. The snapshotter fetches the TOC and verifies this digest against the already-verified manifest. Each regular file and chunk also has a `chunkDigest` in the TOC. The snapshotter uses it to verify each chunk as it arrives.

Fetching one file at a time can add latency. eStargz reduces this cost by placing

prioritized files at the beginning of the layer blob. A `.prefetch.landmark` entry marks the end of that region. The snapshotter can fetch the complete prioritized range with one Range Request before the container starts. The stargz snapshotter identifies prioritized files by running the workload once in a sandbox and recording every file it accesses.

Two client-side components implement eStargz support. The `ctr-remote` command-line interface provides commands for pulling and running eStargz images. The `containerd-stargz-grpc` daemon implements the remote snapshotter used by containerd.

eStargz preserves compatibility with OCI registries by changing the internal layout of each layer. Other systems reduce startup latency through different storage and distribution designs. Slacker [27] serves image contents lazily from a shared backing store. DADI [34] combines fine-grained block transfer with prefetching and peer-to-peer distribution. Starlight [8] jointly redesigns the image format, filesystem, and deployment protocol for edge and wide-area environments. FaaSNet [61] uses an adaptive distribution tree for serverless bursts.

2.5 Two-Dimensional Filesystems

2DFS adds the `2dfs.field` layer type to the OCI image model [2–4]. A field is a sparse two-dimensional matrix of independently addressable *allotments*. Each allotment represents a self-contained, non-overlapping region of the filesystem. For example, one allotment can hold a single model split.

Each allotment references a gzip-compressed tar archive containing its filesystem contents. Allotments contain non-overlapping filesystem paths and can therefore be applied in any order. The same allotment can be referenced by multiple images, which enables content sharing and cache reuse. Allotments can also be serialized as standard OCI layers. Listing 2.3 shows the record stored for each allotment.

The 2DFS builder extends a base OCI image according to a `2dfs.json` descriptor. The descriptor assigns source files, destination paths, and a grid position to each allotment. The builder packages each allotment independently and records its position, digest, and diffID in the `2dfs.field` layer.

Each allotment contains an independent set of static files. The builder can therefore process allotments in parallel and rebuild only those whose contents changed. It also avoids the mount-and-extract step described in Section 2.2.

The 2DFS registry extends an OCI-compliant registry with support for the `2dfs.field` media type and a partitioning operator. A puller selects a partition with a tag of the form `image:tag--x1.y1.x2.y2`. The suffix identifies opposing corners of the requested

```
"allotments": [  
  {  
    "row": int,  
    "col": int,  
    "digest": str,  
    "diffid": str  
  }  
]  
"tot_rows": int  
"owner": str
```

Listing 2.3: The baseline `2dfs.field` manifest: each allotment carries its grid position and the two hashes of its blob.

subfield.

The partitioner handles this tag at the manifest endpoint. It retrieves the OCI+2DFS image, decodes the field, and converts the selected allotments into standard OCI layers. It removes the `2dfs.field` layer and updates the config's root filesystem with the selected allotments' diffIDs. The response is a standard OCI manifest. Any containerd client can therefore pull the selected partition without understanding 2DFS.

2.6 Updating Running Containers and Applications

Orchestrators reduce replacement disruption through rolling and blue-green deployments. Both strategies still start a new instance, incur its cold start, and discard the warm state of the previous instance. Checkpoint/Restore in Userspace (CRIU) [17] captures process memory, open descriptors, and connections. It allows a container to stop and resume. Stateful migration uses the same mechanism to move a container with little downtime [40].

Dynamic software updating changes a running program directly. Kitsune, for example, patches executable code and transfers live state to the new version [28]. In contrast, 2DFS-STAR leaves process state and executable code unchanged and refreshes data exposed through the container filesystem.

Many applications can already react to file changes. Uvicorn watches source files with `--reload` and restarts workers after a change [57]. Jekyll rebuilds affected output with `--watch` and can update a browser through `--livereload` [30]. ML serving systems can also reload weights without restarting the serving process. vLLM, for example, can reload model weights in a running engine [59].

Table 2.1: Capabilities provided by Docker, eStargz, 2DFS, and 2DFS-STAR.

	Independent build	Client-selected partition	Lazy loading	In-place refresh
Docker	no	no	no	no
eStargz	no	no	yes	no
2DFS	yes	yes	no	no
2DFS-STAR	yes	yes	yes	yes

These mechanisms depend on updated data becoming visible in the container filesystem. Current image workflows do not connect a new registry version to an already mounted read-only layer. 2DFS-STAR provides that missing path.

2.7 Model Distribution in Distributed Machine Learning

ML makes image distribution especially costly because model artifacts are both large and short-lived. Systems can retrain and replace models every minute or every few hours [5, 31, 52]. Large models are also often split across devices, with each device holding only part of the network [2, 66]. Both patterns increase the number of distinct image contents that a deployment must build, store, and transfer.

Containers remain the default packaging format for ML models because serving frameworks use them to provide reproducible runtimes across heterogeneous hardware. Model weights are commonly packaged in the image, mounted from external storage, or retrieved by the application at runtime. Packaging weights produces a self-contained artifact, but each weight update requires a new image. External storage separates model and image lifecycles but introduces a separate storage dependency. Application-managed downloads also separate these lifecycles, but move distribution and caching outside the container runtime.

The registry already provides content addressing, caching, and distribution for packaged weights, but conventional layers bind those weights to the rest of the image. 2DFS makes model components independently addressable within the same registry workflow. 2DFS-STAR extends that path with lazy access and in-place updates.

2.8 Capability Comparison

Table 2.1 summarizes the resulting capability gap. Docker provides none of these capabilities. eStargz adds lazy loading, and 2DFS adds independent builds and client-

selected partitions. 2DFS-STAR combines both with in-place refresh. A container can therefore start before all image contents are local and receive updated mounted data without losing its in-memory state.

3 System Design

This chapter presents the design of 2DFS-STAR. It first derives functional and non-functional requirements from the thesis motivation and the characteristics of 2DFS and eStargz. It then uses these requirements to motivate the main design decisions behind 2DFS-STAR builder, 2DFS-STAR registry, and the client-side container stack.

3.1 Functional Requirements

3.1.1 Build

The build process must combine the two image formats in a single pipeline. The resulting image must support both 2DFS partitioning and stargz lazy loading, without requiring a separate conversion pass.

Single-pass build. 2DFS-STAR builder shall produce a single image that is both 2DFS allotment-addressable and stargz lazy-loadable, without a subsequent conversion pass.

Tunable build parameters. 2DFS-STAR builder shall expose the gzip compression level and stargz chunk size as build arguments.

Zstandard compression. 2DFS-STAR builder shall support zstd as a faster alternative for compressing stargz layers.

3.1.2 Distribution

During distribution, 2DFS-STAR must distinguish between the content available to a client and the subset fetched eagerly. At the same time, it must preserve the existing serving behavior of 2DFS and OCI registries.

Lazy client pull. Stargz-compatible clients shall be able to pull 2DFS-STAR images lazily according to the existing stargz specification. The manifest shall retain every layer, including those outside the partition selected by the allotment tag.

Allotment-driven prefetch. The allotment tag shall determine which layers receive prefetch annotations without restricting the layers included in the resulting image.

Backward-compatible registry. 2DFS-STAR registry shall continue to serve non-stargz images and preserve the existing 2DFS allotment-delivery behavior for them.

Annotation preservation. When serving stargz images, 2DFS-STAR registry shall preserve their eStargz manifest annotations.

3.1.3 Update and Refresh

2DFS-STAR must be able to update a running container without restarting it. A refresh must transfer only the content that changed, across all affected layers, and may be triggered either explicitly or by the daemon.

In-place image refresh. 2DFS-STAR shall update a running container to a newer image version without stopping it, provided that the new version retains the same layer count and order.

Change-scoped reload. A refresh shall reload only content that differs between the two image versions, leaving unchanged files untouched. The new content shall remain available for lazy access after the refresh.

Optional background fetch. A refresh shall provide an option to fetch changed and added chunks in the background after the swap.

Refresh subscription. The stargz snapshotter daemon shall support optional polling of a registry tag. When polling detects new content, the daemon shall initiate a refresh.

3.2 Non-functional Requirements

3.2.1 Performance

The performance requirements relate each operation to its relevant baseline. A 2DFS-STAR build should remain close to the cost of a plain 2DFS build, rebuilding should be limited to updated content, lazy pulls should improve on eager pulls, and a refresh should transfer only changed data.

Bounded build overhead. The build overhead of 2DFS-STAR relative to a plain 2DFS build shall not exceed the cost of stargz layer conversion.

Change-proportional rebuild. After a source change, 2DFS-STAR builder shall recompute only the affected allotments, keeping the rebuild cost proportional to the size of the change.

Reduced cold-start latency. For workloads that access only part of the image at startup, the latency from pull to container ready should be lower than that of an eager pull.

Change-proportional refresh. A refresh shall transfer only the changed chunks, keeping its transfer cost proportional to their total size.

3.2.2 Resource Efficiency

Building and serving a 2DFS-STAR image must also have predictable resource costs. In particular, 2DFS-STAR builder should keep its memory usage bounded as image size grows, and 2DFS-STAR registry should add little storage overhead compared with a plain OCI registry.

Bounded builder memory. 2DFS-STAR builder shall stream layer data rather than buffer entire layers, keeping its memory usage bounded regardless of image or layer size.

Low registry overhead. 2DFS-STAR registry's storage overhead should remain close to that of a plain OCI registry.

3.3 System Architecture

2DFS-STAR is a system for building, distributing, and updating container images. It combines the allotment-based partitioning of 2DFS with chunk-based lazy fetching on the client. It also adds in-place image refresh, allowing a running container to adopt a new image version without restarting.

Figure 3.1 follows an image through the build, distribution, and update stages. ① The 2DFS builder takes a standard OCI image and a `2dfs.json` descriptor. It produces an OCI+2DFS image whose `2dfs.field` layer indexes the content as allotments on a grid. 2DFS-STAR extends this step by appending an `eStargz` table of contents to each allotment layer during the build, making every allotment lazy-loadable. ② The resulting image is pushed to the 2DFS registry. ③ At pull time, the registry flattens the allotments into standard OCI layers. 2DFS-STAR includes every allotment in this process, which keeps all layers available for lazy access. It also assigns each layer a `stargz.prefetch` value: the value is nonzero for layers selected by the tag and zero for all others. The

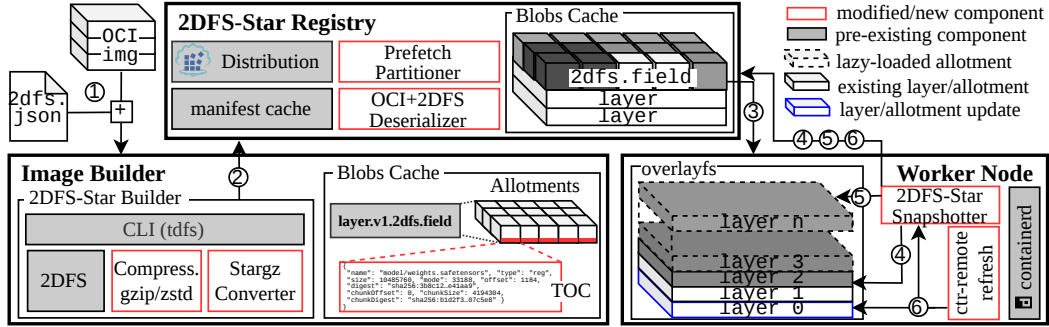


Figure 3.1: 2DFS-STAR architecture. 2DFS-STAR builder produces an OCI+2DFS image, 2DFS-STAR registry serves it with per-layer prefetch annotations, and the client stack pulls, mounts, and refreshes it. New and modified parts are highlighted relative to base 2DFS and the stargz snapshotter.

stargz snapshotter daemon (containerd-stargz-grpc) reads this annotation and treats the two groups differently: ④ layers inside the requested partition are fetched eagerly when the container starts, whereas ⑤ the remaining layers are fetched on demand. ⑥ The `ctr-remote` command-line interface (CLI) controls live updates. It can either refresh a running container directly or subscribe to a registry tag and initiate a refresh when a new version becomes available.

The design therefore extends two 2DFS components as well as the stargz-snapshotter project. The 2DFS builder gains a single-pass stargz build that produces lazy-loadable layers without a separate conversion stage. The 2DFS registry adds per-layer prefetch annotations derived from the allotment tag in the pull request. Finally, the stargz snapshotter daemon gains in-place refresh and subscription support, exposed through corresponding `ctr-remote` subcommands.

3.3.1 Combining Partitioning with Lazy Loading

2DFS and eStargz address complementary parts of the problem. With 2DFS, parts of an image can be built and cached independently, then selected through a semantic tag. However, after the registry serializes a partition, its layers are still pulled eagerly, and the client cannot access content outside the selected partition. eStargz allows any layer to be loaded lazily at chunk granularity, but it has no concept of independently addressable components. Nor can a client indicate at pull time which parts of the image matter most. 2DFS-STAR combines the two approaches by reinterpreting the 2DFS semantic tag. The registry returns all allotments, keeping every file available

for lazy access, and uses the tag only to identify which allotments the client should prefetch. We refer to this approach as *tag-driven prefetch*.

In contrast, eStargz determines its prefetch set at build time. Its toolchain runs the image in a separate optimization pass to record file-access patterns, then marks the accessed files with a prefetch landmark that the snapshotter follows at pull time. This pass must be repeated whenever the image changes and can be costly for frequently updated images or images containing large artifacts. 2DFS-STAR moves the decision from build time to pull time. A single image can therefore serve different clients, each of which selects its prefetch set through the tag, without a new optimization pass after every update.

3.3.2 Image Format

A 2DFS image keeps its top-level layout: an OCI manifest, a config blob, ordinary filesystem layers, and a single `2dfs.field` layer that lists every allotment. 2DFS-STAR extends each allotment record with three optional fields, shown in Listing 3.1. The first four allotment fields are inherited from 2DFS; the three new ones (`tocDigest`, `compression`, `compressedSize`) are absent for legacy gzip allotments and present for stargz allotments.

```
{
  "allotments": [
    {
      "row": int,
      "col": int,
      "digest": str,
      "diffid": str,

      // 2DFS-Star additions (all optional)
      "tocDigest": str,
      "compression": str,
      "compressedSize": int
    }
  ],
  "tot_rows": int,
  "owner": str
}
```

Listing 3.1: `2dfs.field` schema in 2DFS-STAR.

The eStargz table of contents is stored within the allotment layer as the final entry of the compressed tar, and the new `tocDigest` field refers to it. The layer's media type reflects the compression used: `application/vnd.oci.image.layer.v1.tar+gzip` for

legacy and stargz/gzip allotments, `application/vnd.oci.image.layer.v1.tar+zstd` for stargz/zstd ones.

When 2DFS-STAR registry serves a stargz-enabled image, it copies the TOC digest and a prefetch hint into the OCI manifest as per-layer annotations (Listing 3.2). The TOC annotation tells the snapshotter where to find the table of contents in the layer.

```
containerd.io/snapshot/stargz/toc.digest  
containerd.io/snapshot/remote/stargz.prefetch
```

Listing 3.2: Per-layer OCI annotations emitted by 2DFS-STAR registry for stargz-enabled images.

3.3.3 2DFS-Star Builder

2DFS-STAR builder emits a single image that is both allotment-addressable and lazy-loadable. The standard eStargz toolchain instead uses a separate pass that re-reads and re-compresses every layer after the build; for large images, this pass can dominate build time. Integrating the conversion into the allotment build avoids this additional pass and allows the conversion work to run in parallel across allotments. Furthermore, a content cache keyed by file digest allows unchanged allotments to be reused during a rebuild. An update therefore rebuilds only the allotments that changed, unlike stargz builds in which a changed layer invalidates all layers above it.

3.3.4 2DFS-Star Registry

2DFS-STAR registry converts a 2DFS manifest into an OCI manifest at pull time. Two configuration flags control its behavior: one enables stargz serving and the other enables the prefetch annotations.

When stargz support is off, 2DFS-STAR registry preserves the baseline 2DFS behavior: only the allotments inside the requested partition are served, and no stargz annotations are added. When stargz support is on, the partition no longer restricts the layer set. Every stargz allotment in the field is included so that the client can reach any layer lazily, and the partition only drives the prefetch decision: stargz layers inside the partition receive a prefetch value equal to their compressed size, while layers outside it receive a value of zero that disables prefetch for that layer.

The registry must emit layers in a deterministic order because refresh pairs layers by position across image versions. Equivalent pull requests must therefore produce the same layer order and manifest digest.

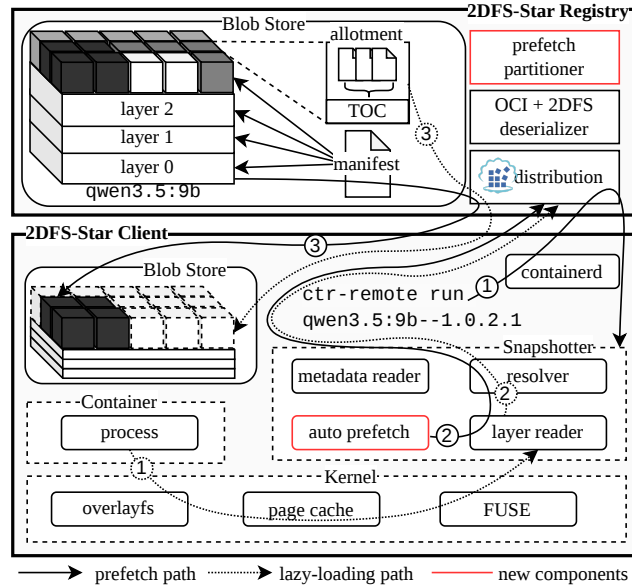


Figure 3.2: Data distribution in 2DFS-STAR with prefetch and lazy loading. Solid arrows trace the prefetch path for the requested partition; dotted arrows trace an on-demand read of a file outside it.

3.3.5 Client-Side Stack

The client-side stack consists of two components. The `ctr-remote` CLI is the user-facing entry point for image pulls, container starts, and image refreshes. It forwards these to `containerd-stargz-grpc`, the `containerd` snapshotter plugin that mounts each image layer through FUSE and serves its files on demand. Because lazy loading takes place on the client and baseline 2DFS provides no client-side component, 2DFS-STAR builds on the maintained `stargz-snapshotter` project.

During a pull, the daemon fetches each layer’s TOC eagerly, while file contents are retrieved on demand through the FUSE mount. Prefetching is disabled by default, which results in a fully lazy pull, but clients may enable tag-driven prefetch instead.

Figure 3.2 illustrates both paths using a partitioned pull of the image `qwen3.5:9b--1.0.2.1`. Along the prefetch path, ① `ctr-remote` asks `containerd` to pull the reference; `containerd` looks for the manifest in its local blob store and, on a miss, requests it from 2DFS-STAR registry. There the OCI distribution front end decodes the `2dfs.field`, and the prefetch partitioner serializes every allotment as an OCI layer. Tag-driven prefetch then marks the selected subfield (the cells from `<1,0>` to `<2,1>`) for eager fetch and leaves the rest

lazy. ② The daemon's auto-prefetch component reads these annotations and asks the layer reader to pull the tagged layers. ③ The layer reader fetches them from the registry into the client's blob store in the background. The container can therefore start without waiting for the transfer to finish and read the prefetched bytes locally once they arrive.

The lazy path, shown with dotted arrows, handles files outside the selected partition on demand. ① When the container reads a file that is not available locally, the request passes through overlays and the page cache before reaching FUSE. ② FUSE forwards the request to the snapshotter. Using the TOC, the metadata reader identifies the file's layer and chunks. The layer reader then determines that the data is absent from the client blob store. ③ The resolver fetches the missing chunks from the registry. The layer reader caches the chunks and returns the data to the process, so subsequent reads can be served locally.

Image refresh uses the same machinery, as shown in Figure 3.3. ①a A `ctr-remote refresh` command or ①b a subscription poll initiates the refresh. The CLI reads the old manifest from containerd's local image record, locates the mounted version of the image, and resolves the new manifest from the registry. Matching manifest digests indicate that the image has not changed, so the refresh becomes a no-op. If the digests differ, an update is available.

② Before applying the update, the daemon checks that the two manifests have the same layer count, layer order, and media types. It rejects structurally incompatible updates because the lower-layer set of the container's overlay mount is fixed when the container starts. The daemon then pairs layers by position and skips pairs with identical digests. For each changed pair, it fetches the new table of contents, verifies it against the TOC digest in the manifest, and compares it with the mounted TOC at chunk granularity. Each chunk is classified as unchanged, changed, or added. If the two tables cannot be aligned entry by entry, the daemon re-reads the entire layer instead.

③ For each changed layer, the daemon replaces the reader behind its FUSE handles with one backed by the new blob. Reads already in progress remain safe: a read that acquired the old reader before the swap continues to use the old blob. The blob is released only after all such readers have finished, preventing failures caused by a blob disappearing during a read. ④ The daemon then invalidates only the page-cache ranges that correspond to changed chunks. FUSE fetches the new data on the next access, while unchanged files remain warm in the page cache. By default, changed chunks are fetched lazily; an optional flag fetches them in the background immediately after the swap so that later reads do not wait for the network.

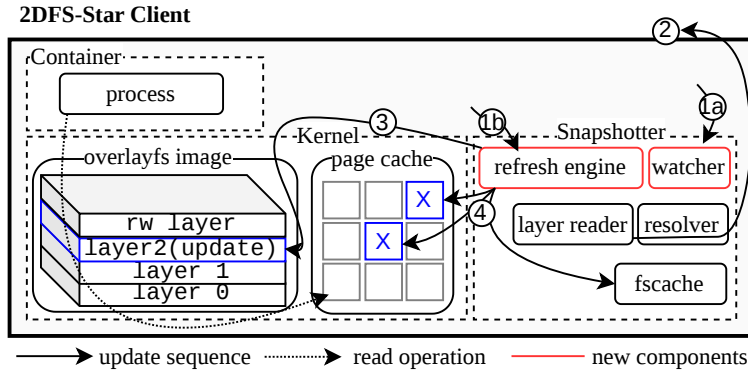


Figure 3.3: In-place refresh and subscription in 2DFS-STAR. A refresh is triggered by an explicit command or by a subscription poll; the daemon diffs the changed layers at chunk granularity, swaps the layer readers, and invalidates only the changed cache ranges.

3.3.6 Subscription Mode

A refresh still requires a trigger. Subscription mode places this responsibility in the stargz snapshotter daemon. After a client subscribes to a registry tag, the daemon refreshes the container whenever that tag points to new content. This allows a workload to follow a moving tag without an external orchestrator. The `ctr-remote` CLI exposes the three subcommands in Listing 3.3: one starts a subscription, one stops it, and one lists the active subscriptions.

```
ctr-remote watch <ref> [--interval <d>] [--with-background-fetch]
ctr-remote unwatch <ref>
ctr-remote watch-list
```

Listing 3.3: Subscription subcommands added to `ctr-remote`.

A watch request contains both the image reference and the list of layer digests held by the client. The daemon retains this list as part of the subscription state. It therefore does not need a global index that maps image references to layers: the poll loop already has the layer set required to compare a new version with the mounted one.

Because registries provide neither push notifications nor long polling for tag updates, the daemon checks for new versions periodically. It runs one poll loop per subscription, with a configurable interval of 30 s by default. Each iteration resolves the tag using a single HEAD request. If the returned top-level digest matches the previously observed digest, the iteration ends without fetching the manifest body. A subscription with no updates therefore costs one HEAD request per interval. If the digest has changed,

the daemon fetches the manifest body, follows one index level for a multi-platform reference, and initiates the same per-layer delta refresh as the manual command. The subscription state stores both the top-level resolved digest, used for the early exit, and the platform manifest digest reported by `watch-list`. These digests are identical for a single-platform image but differ when the reference points to an index.

A subscription ends when the client unwatches it, the watched layers are unmounted, ten consecutive polls fail, or the daemon restarts. After the tenth failed poll, the daemon removes the subscription and emits a warning. Subscriptions are held only in daemon memory, so clients must register them again after a restart. A new watch request for an already watched reference replaces the existing subscription. Persisting subscriptions across restarts and quiescing a busy workload before refresh are outside the scope of this thesis.

3.3.7 Default Fallback Path

Images built without `stargz` continue to use the original 2DFS pull-and-run path. The `--enable-stargz` build flag selects between the two modes. When the flag is absent, 2DFS-STAR builder writes plain gzip allotments, 2DFS-STAR registry applies the original partition filter, and any OCI-compliant client pulls the result eagerly. Thus, the same builder, registry, and client toolchain support both lazy and eager operation without requiring separate deployments.

4 Implementation Details

The implementation of 2DFS-STAR spans three codebases. 2DFS-STAR builder creates lazy-loadable allotments, 2DFS-STAR registry serves them, and the stargz snapshotter loads and refreshes them on the client. Each component extends an existing system. This chapter follows an image through these components during build, pull, and refresh, explaining how the design presented in Chapter 3 is implemented.

4.1 2DFS-Star Builder

To make an allotment lazy-loadable, 2DFS-STAR builder writes it in the seekable eStargz format rather than plain gzip (Subsection 3.3.3). The `--enable-stargz` flag enables this behavior. When the flag is omitted, the builder retains the original 2DFS behavior and produces a gzip-compressed allotment.

The build pipeline branches according to the selected compression mode. On the gzip path, 2DFS-STAR builder computes the diff identifier from the uncompressed tar and writes a standard gzip blob. On the stargz path, it delegates compression to the eStargz library. Because the library rebuilds the tar and its output is not byte-identical to the input, the builder obtains the diff identifier from the resulting blob. The blob is streamed into the cache and hashed as it passes through the pipeline; the complete layer is never buffered in memory. Builder memory usage therefore remains bounded regardless of layer size. The `--use-zstd` flag selects chunked zstd as the compression format within the stargz path.

The `tdfs build` command exposes the six compression parameters listed in Table 4.1. At build time, each allotment is divided into fixed-size chunks. The default size is 4 MiB, matching the eStargz default. Chunk size and compression level determine the trade-off between build time and transfer size. Larger chunks reduce per-chunk overhead but make subsequent lazy fetches and refreshes less granular. A higher compression level reduces blob size at the cost of a longer build.

2DFS-STAR stores its build cache on disk in the directory selected by `--home-dir`, which defaults to `~/2dfs`. Separate subdirectories hold compressed blobs, cache keys derived from uncompressed tars, and manifest index entries. Each cache key includes the TOC digest, allowing the builder to distinguish stargz and gzip representations of the same files.

Table 4.1: Build-time compression flags on `tdfs build`.

Flag	Default	Purpose
<code>--enable-stargz</code>	<code>false</code>	eStargz instead of gzip
<code>--stargz-chunk-size</code>	4 MiB	eStargz chunk size
<code>--stargz-compression-level</code>	<code>gzip.BestSpeed</code>	gzip level in stargz
<code>--use-zstd</code>	<code>false</code>	zstd:chunked in stargz
<code>--zstd-compression-level</code>	<code>SpeedFastest</code>	zstd encoder level
<code>--compression-level</code>	<code>gzip.BestSpeed</code>	legacy gzip level

4.2 2DFS-Star Registry

The registry changes are confined to `ConvertTdfsManifestToOciManifest`, which converts the `2dfs.field` into OCI layer descriptors at pull time. The `StargzSupport` configuration option enables the extended conversion path. `PrefetchSupport` enables prefetch annotations and is valid only when stargz support is active.

The conversion iterates over the allotments and creates one descriptor for each distinct layer digest. For a stargz allotment, it copies the stored TOC digest into the descriptor's `containerd.io/snapshot/stargz/toc.digest` annotation. An allotment may occur in more than one partition, but the resulting manifest contains it only once. When prefetch support is enabled, its prefetch value is initially zero and is set to its compressed size if any occurrence belongs to the requested partition.

Finally, the conversion emits the descriptors in digest-sorted order. This removes any dependence on map-iteration order and ensures that equivalent requests produce the stable layer order and manifest digest required by Subsection 3.3.4.

4.3 Client-Side Stack

The client-side implementation consists of the `ctr-remote` CLI and the `containerd-stargz-grpc` daemon (Subsection 3.3.5). 2DFS-STAR extends this stack with an in-place refresh path and a subscription loop that triggers it.

4.3.1 Snapshotter Configuration

At startup, the stargz snapshotter daemon reads its settings from `config.toml`. These settings govern caching, prefetching, chunk verification, and the handling of reads whose data is already local. This section covers only the options that affect how 2DFS-

STAR loads and serves an image. They are existing stargz features that 2DFS-STAR uses without modification.

Prefetch

Prefetch downloads a range of a layer blob into the local cache before the container accesses it. Files within this range can then be served from disk rather than fetched from 2DFS-STAR registry on first access. The range extends from the beginning of the layer to a cut-off offset. The daemon determines this offset from three possible inputs, evaluated in a fixed order.

At build time, eStargz can encode the cut-off using a landmark file. A layer optimized for a particular workload contains a `.prefetch.landmark` entry at the boundary between the files accessed by that workload and the remaining files. A layer built without this optimization contains a `.no.prefetch.landmark` entry instead. When mounting the layer, the daemon first checks its table of contents for these entries. A no-prefetch landmark disables prefetch, whereas a prefetch landmark sets the cut-off to the landmark's offset. Only when neither entry is present does the daemon use the third input: the byte offset stored in the image manifest's prefetch annotation.

2DFS-STAR builder disables landmark insertion, so the daemon always obtains the range from the prefetch annotation. 2DFS-STAR registry writes this annotation separately for each layer at pull time (Section 4.2). Consequently, the requested partition determines the prefetch range during the pull; the build does not impose a fixed boundary.

By default, container launch waits for prefetch to complete. This delay is bounded by `prefetch_timeout_sec`, which defaults to ten seconds. Once the timeout expires, the container starts even if prefetch is still running. The wait can also be avoided by setting `prefetch_async_size` to a nonzero value below the prefetch range. In this mode, the daemon starts the container immediately and completes the transfer in the background. The default value of zero retains synchronous prefetch. Finally, `noprefetch` disables prefetch entirely, leaving the layer fully lazy until file reads request its chunks.

Background Fetch

Background fetch downloads an entire layer into the local cache while the container is running. Once a layer is mounted, the daemon fetches its blob chunk by chunk, even if the container never reads it. Unlike prefetch, which runs in the foreground before container launch, background fetch runs at low priority. It yields to prefetch and on-demand reads; if foreground work arrives during a fetch, the background operation is interrupted and resumes afterwards. Each layer is processed once, with

data streamed directly to disk. The `no_background_fetch` option disables this behavior.

Verification

During streaming, the daemon verifies each chunk against its recorded digest (Section 2.4). Setting `disable_verification` disables this check.

FUSE Passthrough

FUSE passthrough changes how the snapshotter serves cached files. This feature was introduced in Linux 6.9 and adopted by eStargz; 2DFS-STAR uses it without modification. Without passthrough, every read of a mounted file reaches the daemon, which locates and decompresses the corresponding chunks before returning the requested bytes. With passthrough enabled, the kernel reads an already-local file directly, avoiding a round trip through the daemon for each read. Warm files can therefore be read at native filesystem speed.

4.3.2 Delta Refresh

In-place refresh (Subsection 3.1.3) updates a running container to a newer image version by fetching only the chunks that differ. The implementation has three main steps: comparing the old and new tables of contents for each layer, swapping the layer reader under a lock, and invalidating the affected cache entries.

For every layer whose descriptor has changed, the daemon compares the new table of contents with the mounted one in two stages (Listing 4.1). The first stage checks whether the tables have the same structure. They must contain the same number of files and directories, and each entry must have the same name and parent directory in both versions. Adding, removing, moving, or renaming an entry breaks this alignment. In that case, the daemon falls back to refreshing the entire layer lazily.

If the structures align, the second stage compares chunks at matching file identifiers and offsets. A chunk is classified as changed when its digest or size differs, and as added when its offset exists only in the new table. All remaining chunks are unchanged and continue to use their existing cache entries.

DeltaResult:

```
Fallback bool // TOCs cannot be aligned: refresh the whole layer
ChangedChunks []Chunk // same (fileID, offset), digest or size differs
AddedChunks []Chunk // offset present only in the new TOC
StaleCacheKeys []string // cache keys to drop
ChangedFiles []fileID // touched files, for scoped FUSE invalidation
```

```
# a path is one file or directory
# fallback means treat the whole layer as changed
diff(oldTOC, newTOC):
    if pathCount(oldTOC) != pathCount(newTOC):
        return fallback
    for each pathId in oldTOC:
        if (parent, name) differ across TOCs:
            return fallback
    for (fileId, offset) in newTOC chunks:
        newChunk = newTOC chunk at (fileId, offset)
        oldChunk = oldTOC chunk at (fileId, offset) # null if absent
        if newChunk has no digest:
            return fallback
        else if oldChunk is null:
            append newChunk to AddedChunks
        else if oldChunk has no digest:
            return fallback
        else if newChunk digest or size differs from oldChunk:
            append newChunk to ChangedChunks
            append oldChunk's cache key to StaleCacheKeys
        # otherwise unchanged: serve from cache
    for each file with a changed or added chunk:
        append its file cache key to StaleCacheKeys
        append it to ChangedFiles
```

Listing 4.1: Per-layer table-of-contents comparison. A structural mismatch triggers a whole-layer refresh; otherwise, chunks are classified by file and offset, and the affected files are recorded for invalidation.

The amount of transferred data therefore depends on the changed chunks rather than the size of the layer. If rebuilding a layer of n chunks modifies only one chunk, the daemon fetches that chunk alone. The remaining $n - 1$ chunks continue to be served from the on-disk cache.

Cache reuse across image versions requires unchanged chunks to retain their cache keys. Each key contains a file identifier assigned while the daemon traverses the table of contents, so this traversal must produce stable identifiers. In the original implementation, directory children were visited in an order that could vary between builds. Consequently, even identical tables of contents could assign different identifiers to the same file and miss valid cache entries. 2DFS-STAR makes the traversal deterministic by sorting the children of each directory by name. File identifiers then depend only on the names and shape of the tree. Changing one file's contents leaves the identifiers of all other files unchanged, keeping their cached chunks accessible.

Whether the daemon can perform a chunk-level refresh instead of falling back to

the complete layer also depends on the order of entries in the source tar. 2DFS-STAR builder and common container builders emit entries in a stable, sorted order, allowing content-only changes to preserve identifier alignment. Adding or removing a file, or reordering the tree, can shift the identifiers and trigger the structural fallback. The daemon then re-reads the complete layer. Cache entries that remain under their old keys become unreachable until the layer is unmounted. This may increase bandwidth use but does not affect correctness, because every cache miss causes the data to be fetched again.

Each mounted layer has a reader that its FUSE nodes use to locate, fetch, and decompress chunks. 2DFS-STAR stores this reader in a per-layer reference protected by a read-write lock. During refresh, the daemon creates a reader for the new blob and replaces the reference atomically. The old blob remains available until all reads that use it have completed. A read that started before the swap can therefore finish using the old blob, while subsequent reads use the new reader.

After the reader swap, the daemon invalidates data at two levels. It first removes the on-disk cache entries for changed chunks while retaining all unchanged entries. It then invalidates the corresponding ranges in the page cache, as required by the scoped invalidation in Subsection 3.1.3. When `--with-background-fetch` is set, the daemon fetches the changed chunks in the background immediately after the swap. Otherwise, they are fetched on their next access.

Refresh modifies only the daemon's in-memory layer state. The image record in containerd continues to refer to the original digests, so `ctr images ls` reports the image version from before the refresh. Keeping this record consistent would require updates through containerd's image service, including handling failures in that process, and is outside the scope of this work.

4.3.3 Integration

Refresh makes the updated bytes available through the paths already visible to the container. Whether an application observes those bytes immediately depends on how it accesses the file. 2DFS-STAR does not automatically reload application-managed state.

Two access patterns observe an update without additional application logic. First, a process may access the file through the read system call. The scoped invalidation described in Subsection 4.3.2 removes changed ranges from the page cache, so the next read of such a range retrieves the updated bytes through FUSE. Second, a process may use a memory-mapped file. For example, when `torch.load` is called with `mmap=True`, invalidation removes the changed pages from the mapping. Accessing those addresses again triggers a page fault and retrieves the new bytes through FUSE.

By contrast, a process that has copied file data into application memory must respond

to the refresh explicitly. Page-cache invalidation cannot update bytes already stored in a buffer or weights parsed into application-specific structures. The application must therefore re-read the data in response to a suitable trigger, such as a reload endpoint, signal, or file-system watch.

4.3.4 Subscription Mode

Subscription mode is implemented as a scheduling layer around the manual refresh path. Each active subscription owns a goroutine driven by a `time.Ticker`. When the digest comparison described in Subsection 3.3.6 detects a new image version, the goroutine passes the subscription's layer set to the same delta-refresh function used by the manual command (Subsection 4.3.2).

5 Evaluation

We evaluate 2DFS-STAR at four stages of the container image lifecycle. The experiments cover initial builds, rebuilds after an update, image pulls followed by first workload access, and live refresh of a running container. We compare the cost of combining independent allotments with lazy loading against conventional OCI images, 2DFS without lazy loading, and eStargz.

5.1 Experimental Setup

We run the full set of experiments in a cloud environment, then repeat the pull and refresh measurements at the edge to assess performance under tighter resource constraints.

Cloud environment. The cloud environment uses two Dell PowerEdge c6525-25g nodes. One node runs 2DFS-STAR builder and the client stack. The other hosts 2DFS-STAR registry. Each node has one 16-core AMD EPYC 7302P processor with 32 hardware threads, 128 GB of memory, two 480 GB Serial ATA (SATA) solid-state disks, and a 25 GbE network interface. Both are CloudLab [20] bare-metal machines running Ubuntu 24.04.1 on Linux 6.17.0-35-generic. The container stack consists of containerd v2.2.1, runc 1.3.4, BuildKit v0.26.3, and nerdctl 2.2.1. Our stargz snapshotter is based on v0.18.2. No competing workloads run on the nodes during measurement.

Edge environment. The edge client has a dual-core Intel Core i5-6260U with four hardware threads, 16 GB of memory, a 250 GB Samsung 850 EVO SATA solid-state disk, and a 1 GbE interface. It runs Ubuntu 24.04 on Linux 6.17.0-29-generic. 2DFS-STAR registry runs on a four-vCPU virtual machine with 8 GB of memory, a 100 GB virtual disk, and a 10 GbE interface. The registry machine runs Ubuntu 24.04.4 on Linux 6.8.0-106-generic. The institution network connects the two machines. Their round-trip time averages 0.408 ms over 40 Internet Control Message Protocol (ICMP) probes.

Snapshotter configuration. Every client workload uses the stargz snapshotter configuration in Listing 5.1. Only the registry mirror host changes between the cloud and edge deployments. We disable prefetch and background fetch because both can compete with the reads performed by the workload. We also disable verification, which removes the fixed cost of hashing each chunk during a read. FUSE passthrough is enabled because it is the fastest path for the read-heavy access this workload drives. The mirror block directs the stargz snapshotter daemon to 2DFS-STAR registry over plain HTTP.

Sections 5.10 and 5.11 evaluate passthrough and prefetch in isolation to confirm that this configuration is the best match for our workload.

```
noprefetch = true
no_background_fetch = true
disable_verification = true

[[resolver.host."10.10.1.2:5000".mirrors]]
host = "10.10.1.2:5000"
insecure = true

[fuse]
passthrough = true
```

Listing 5.1: The stargz snapshotter configuration shared across all measurements.

Baselines. Our benchmarks compare 2DFS-STAR against Docker, eStargz, and 2DFS. Table 2.1 summarizes the capabilities provided by each method. *Docker* represents a conventional OCI image without lazy loading and uses BuildKit. *eStargz* uses the same BuildKit version with eStargz support enabled. This integrated path avoids a separate conversion of an ordinary OCI image. *2DFS* and *2DFS-STAR* use 2DFS-STAR builder with the options for their respective formats. We use Zstandard for 2DFS-STAR in the main experiments because it compresses faster than gzip at comparable ratios. Section 5.9 measures the effect of both compression algorithms in isolation.

Workloads. We use three language model images that span a wide range of sizes and architectures: OpenLLaMA 3B [44], Qwen3.5 9B [45, 65], and Gemma-4 31B [24, 25, 55]. To exercise the multi-model case we add a video analytics bundle of four independent computer vision (CV) models: the ResNet-50 [29] and EfficientNetV2-M [54] classifiers, the YOLOv3 object detector [47, 48], and the DeepLabV3 semantic segmentation model [9, 10]. Table 5.2 lists what each of the four contributes to the bundle. Each image pairs model weights with the runtime that serves them. The three language model images build on the Ollama runtime image (ollama/ollama) [41]. The video an-

Table 5.1: The workloads, the base image each builds on, and the model files it carries. Base image sizes are compressed.

Workload	Base image	Base size	Model size
OpenLLaMA 3B	ollama/ollama	3.38 GB	6.2 GB
Qwen3.5 9B	ollama/ollama	3.38 GB	16 GB
Gemma-4 31B	ollama/ollama	3.38 GB	59 GB
CV bundle (four models)	tensorflow/tensorflow	616 MB	686 MB

Table 5.2: The four models in the video analytics bundle.

Model	Task	Size
ResNet-50	Classification	97.8 MB
EfficientNetV2-M	Classification	204.3 MB
YOLOv3	Object detection	235.4 MB
DeepLabV3	Semantic segmentation	148.2 MB

alytics bundle builds on the TensorFlow runtime image (tensorflow/tensorflow) [56]. Table 5.1 lists what each image carries.

Methodology. Build experiments vary the *split capacity*. This value sets the number of layers or allotments relative to the finest decomposition produced during workload preparation. We cap the capacity to keep the resulting units roughly equal in size. A capacity of 0% stores the model as one unit. A capacity of 50% repackages a model with at most 100 splits into 50 units. All baselines package the same files with the same logical grouping at each capacity. The contents therefore remain fixed while the partitioning changes.

Each image update experiment begins with a baseline image. We then flip one bit in each selected safetensors or JSON file and rebuild the same specification. The bit flip changes the file digest and forces the toolchain to rebuild its unit. The modified units cover 25%, 50%, 75%, or 100% of the image. We select them from the bottom upward or from the top downward. Direction does not affect 2DFS-STAR or 2DFS because their allotments are independent. However, it affects Docker and eStargz because a changed lower layer invalidates every layer above it.

Pull experiments vary the *partition size* from 25% to 100% of the model contents. Docker uses a separate image for each partition size that contains only the selected files. 2DFS uses a single image, but its partition tag returns only selected allotments. eStargz

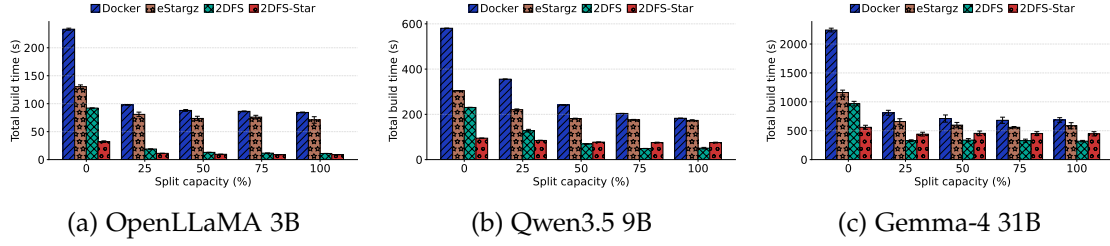


Figure 5.1: Build time of 2DFS-STAR against Docker, 2DFS, and eStargz across three language models, as a function of split capacity.

and 2DFS-STAR also use a single image. The workload reads only selected model files, which the lazy snapshotter fetches on demand. Reported latency includes the image pull and the first read of these files. Stacked bars separate these stages.

Refresh experiments change *tokenizer_config.json* and thereby invalidate one layer. The two manual baselines stop the container, pull the updated image, start a new container, and read the data again. 2DFS-STAR keeps the container running, refreshes its mounted image, and repeats the read.

Unless stated otherwise, each bar reports the median of three to five runs. Error bars show one standard deviation across those runs.

5.2 Build Time

2DFS-STAR builds all three language-model images faster than Docker and eStargz at every split capacity in Figure 5.1. Docker and eStargz take longest at 0% split capacity. At this capacity, each method stores the model as one layer or allotment. Higher capacities divide the model into more units and expose more build work that can run in parallel. 2DFS-STAR also benefits from finer partitioning, but its build time changes less across the tested range. On Qwen3.5 9B, 2DFS-STAR takes roughly 95s with one allotment and about 75s at full capacity. Docker falls from roughly 580 to 180s, while eStargz falls from 305 to 175s.

The comparison with 2DFS shows the cost of making allotments lazy-loadable under the evaluated configurations. 2DFS-STAR is faster at low split capacities. 2DFS overtakes it once enough allotments are available, especially on the larger Qwen and Gemma workloads. The two builds differ in two relevant ways. 2DFS-STAR generates the eStargz metadata required for on-demand reads and uses Zstandard instead of the gzip compression used by 2DFS.

5 Evaluation

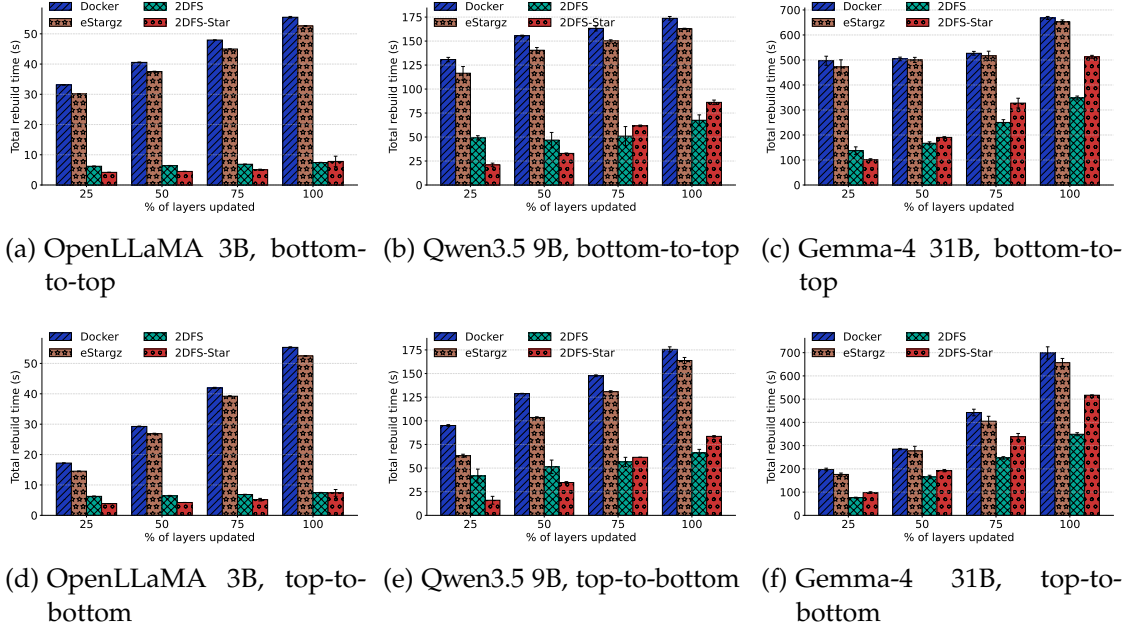


Figure 5.2: Rebuild time against the fraction of modified files, with the top row changing files bottom-to-top and the bottom row top-to-bottom.

5.3 Image Update Time

Independent allotments make rebuild cost proportional to the changed content. Figure 5.2 varies the fraction and position of modified files. Docker and eStargz become slower as more layers change. Their results also depend on update direction. Even a small change near the bottom approaches the cost of a full rebuild because it invalidates the unchanged layers above it. 2DFS and 2DFS-STAR rebuild only the affected allotments. Their cost follows the amount of changed data and remains the same in both directions.

2DFS-STAR responds to changed-data volume from the smallest update we test. It is the fastest method while the update covers less than half of the image. 2DFS overtakes it when the update reaches about three quarters of the model. 2DFS-STAR nevertheless remains much faster than Docker and eStargz throughout. For a 25% bottom-to-top update of Qwen3.5 9B, 2DFS-STAR takes about 21 s. The same update takes roughly 49 s with 2DFS, 117 s with eStargz, and 131 s with Docker. At 100%, 2DFS-STAR takes about 86 s and 2DFS takes about 67 s.

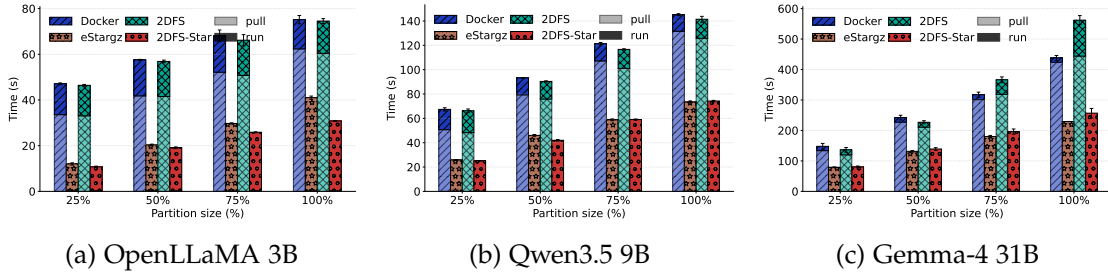


Figure 5.3: Pull plus first-read latency as the application accesses 25% to 100% of each model. The hatched lower segment is the pull, the solid upper segment the read.

5.4 Pull Time and Cold-Start Latency

Docker and 2DFS transfer complete layer contents before the container can use them. Their latency therefore remains nearly constant as the accessed fraction changes in Figure 5.3. eStargz and 2DFS-STAR fetch file contents on demand, and their nearly identical latency grows with partition size. For Qwen3.5 9B, 2DFS-STAR completes the 25% case in about 25 s. Docker and 2DFS take roughly 67 and 66 s. At full access, 2DFS-STAR takes about 74 s and remains close to eStargz. This time is still about half that of the eager baselines because the application does not read every file in the base image. Docker and 2DFS pull the complete base image, while 2DFS-STAR and eStargz leave unused files remote.

5.5 Live Refresh

Live refresh produces the largest difference between 2DFS-STAR and the baselines. The top row of Figure 5.4 compares in-place refresh with replacement of an eStargz or conventional OCI container. Both manual paths stop the running container, pull the replacement image, and start another container. 2DFS-STAR instead updates the mounted snapshot while the original container continues to run. Its critical path contains only the refresh and the on-demand read that follows. Across the three models, this reduces end-to-end update latency by 57.5 to 62.5 times relative to eStargz replacement and by 69.2 to 85.9 times relative to OCI replacement.

The latency gain comes with a much larger reduction in data transferred. A refresh transfers only the changed chunks and the table-of-contents metadata that locates them. The bottom row of Figure 5.4 reports the data transferred by one update. Manual methods transfer the whole image, so their payload grows with the model: 6.4 to 8.0 GB

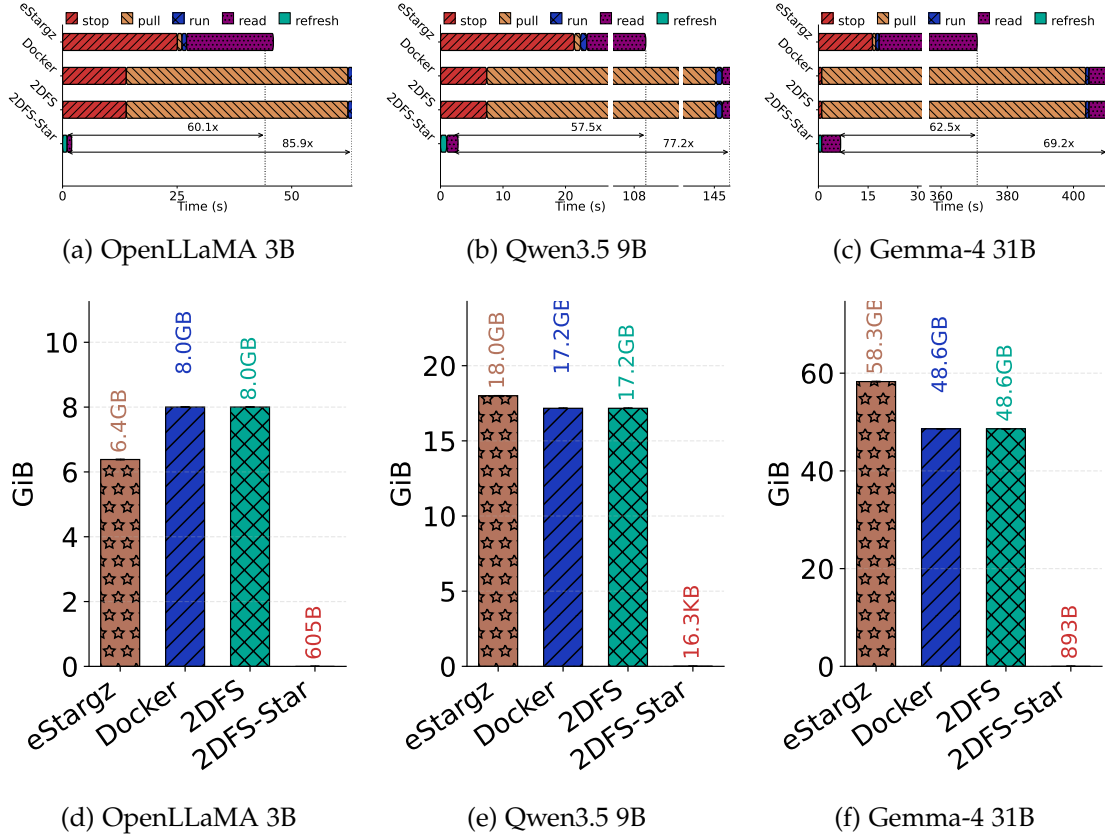


Figure 5.4: Live refresh against manual replacement. The top row is end-to-end update latency; the bottom row is the data transferred during one update.

for OpenLLaMA, 17.2 to 18.0 GB for Qwen, and 48.6 to 58.3 GB for Gemma. 2DFS-STAR transfers 605 B, 16.3 KB, and 893 B, respectively, six or more orders of magnitude less, and its payload does not grow with the model.

5.6 Resource Usage

2DFS-STAR records the lowest cumulative CPU time among the four methods on both the builder and client. We sample processor usage and used memory once per second on the cloud nodes. Summing these samples over a run gives CPU time in core-seconds and memory-time in gigabyte-seconds. Both metrics include elapsed time, which means that faster completion contributes to lower totals.

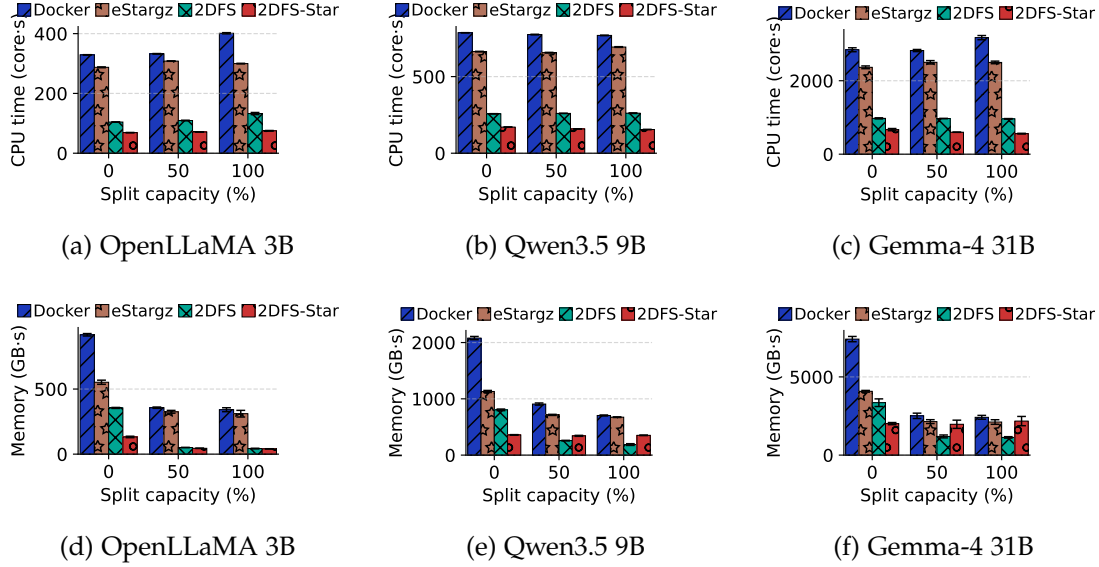


Figure 5.5: Resource usage on the builder against split capacity. The top row is central processing unit (CPU) time in core-seconds, the bottom row memory held in gigabyte-seconds.

5.6.1 Build

Figure 5.5 reports resource usage during build. 2DFS-STAR uses the fewest core-seconds at every split capacity for all three models. At full capacity, it uses 75 core-seconds for OpenLLaMA 3B compared with 133 for 2DFS. The corresponding values are 155 and 261 for Qwen3.5 9B, and 559 and 962 for Gemma-4 31B. 2DFS achieves its shorter elapsed build time (Section 5.2) through greater concurrency. It keeps an average of 5.1 cores busy on Qwen3.5 9B, while 2DFS-STAR uses 2.0. Additional eStargz metadata increases elapsed time without producing a corresponding increase in total CPU time.

A 2DFS-STAR build uses about 1 GB more memory than a 2DFS build. On Qwen3.5 9B, the two use 4.5 and 3.6 GB. On Gemma-4 31B, they use 4.8 and 3.5 GB. This gap remains similar across split capacities. Because gigabyte-seconds include runtime, 2DFS-STAR can record a lower total despite using more memory while the build is running. At 0% capacity, 2DFS-STAR records 358 gigabyte-seconds on Qwen3.5 9B compared with 804 for 2DFS because 2DFS-STAR finishes much sooner. 2DFS records the lower total from 50% capacity on Qwen3.5 9B and from 25% on Gemma-4 31B. At those capacities, 2DFS no longer has a runtime disadvantage. 2DFS-STAR remains below Docker at every capacity and on every model. It also remains below eStargz except at the two finest capacities on Gemma-4 31B, where their results differ by less

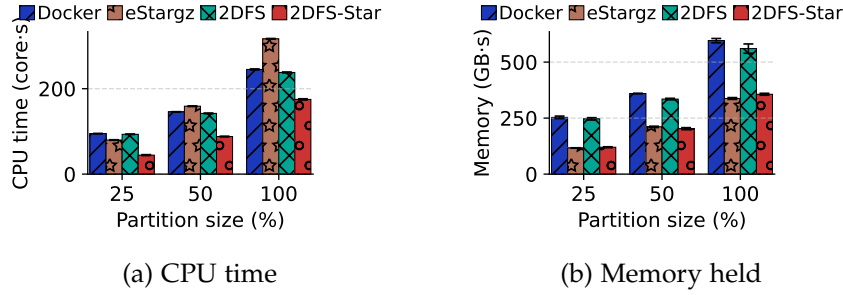


Figure 5.6: Resource usage on the client during pull plus first read of Qwen3.5 9B, against partition size.

than 3%.

5.6.2 Pull

Figure 5.6 reports client resource use during pull and first read. 2DFS-STAR records the lowest CPU time at every partition size. A full read of Qwen3.5 9B uses 175 core-seconds and an average of 2.4 cores. Docker uses 245 core-seconds, while eStargz uses 317 core-seconds and an average of 4.3 cores. The same pattern appears at a larger scale for Gemma-4 31B. 2DFS-STAR uses 664 core-seconds compared with 1161 for eStargz. One contributing factor is the use of `zstd:chunked` in 2DFS-STAR and `gzip` in eStargz. The compression ablation in Section 5.9 shows the resulting difference in pull latency. Every file read during a lazy pull must be decompressed, which makes compression cost recur throughout the read.

Lazy pulls record fewer gigabyte-seconds than eager pulls. 2DFS-STAR and eStargz record 338 and 357 gigabyte-seconds, compared with 596 for Docker and 560 for 2DFS. The two lazy methods remain within a few percent of each other. Average resident memory ranges from 3.3 to 5.1 GB on a node with 128 GB. None of the methods is memory-bound in this experiment. The integrated totals mainly reflect their different runtimes.

5.7 Video Analytics Application

The language-model experiments divide one large artifact. The video analytics bundle represents a different use case: a client chooses among four independent CV models. Its build and rebuild results in Figure 5.7 follow the pattern observed for the language models. Independent allotments make 2DFS and 2DFS-STAR faster to build than Docker

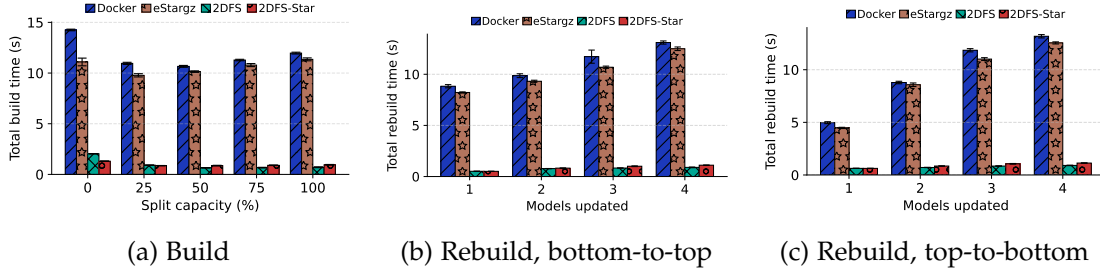


Figure 5.7: Build and rebuild of the video analytics bundle of four independent CV models.

and eStargz. Their update cost grows with the number of modified models. The linear layer stacks also rebuild dependencies invalidated by an edit. 2DFS-STAR remains close to 2DFS while producing lazy-loadable allotments.

Lazy loading is most useful when the application selects only part of the bundle. Figure 5.8 varies the number of models accessed after startup. Lazy methods leave unused models remote, and their advantage is largest when the application needs one or two models. The difference narrows as the application reads the full bundle. Even at full access, 2DFS-STAR and eStargz finish in less than half the time of the eager baselines. Eager paths pull and unpack the complete image regardless of selected models.

The refresh experiment replaces ResNet-50 while leaving the other three models unchanged. 2DFS-STAR refreshes its allotment without stopping the container. In the cloud environment, this is 8.3 times faster than eStargz replacement and 9.3 times faster than Docker or 2DFS replacement. The update transfers 97.9 MiB with 2DFS-STAR, 686.9 MiB with eStargz, and 1.2 GiB with Docker and 2DFS. The 2DFS-STAR payload corresponds to the ResNet-50 model size in Table 5.2. The other methods fetch all four models again.

5.8 Performance at the Edge

Our edge experiments test whether the cloud results carry over to a slower client and network link. Absolute pull times increase, but the ordering in Figure 5.9 does not change. eStargz and 2DFS-STAR avoid eager transfer and retain their largest advantage for partial access. Lazy loading therefore remains effective with four hardware threads and a 1 GbE link. The slower link increases the stop-and-pull interval removed by live refresh. Figure 5.10 shows this effect. For OpenLLaMA, 2DFS-STAR is 141.5 times faster than manual eStargz replacement and 265.6 times faster than OCI replacement. The corresponding factors for Qwen are 12 and 19.3. Qwen’s larger refresh payload

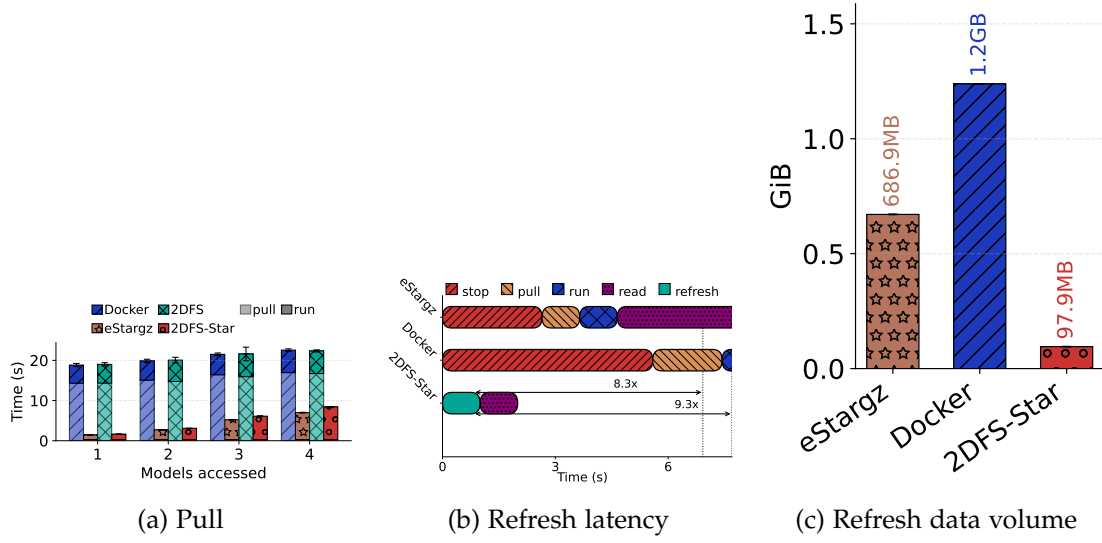


Figure 5.8: Pull and refresh of the video analytics bundle.

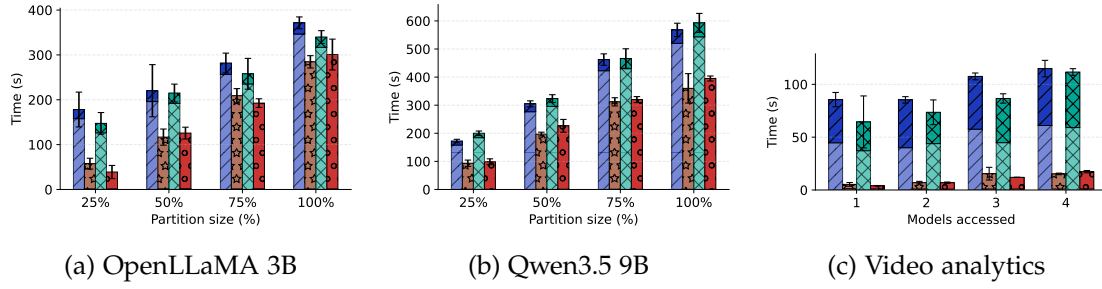


Figure 5.9: Pull performance on the resource-constrained edge client.

(Section 5.5) reduces its relative gain, but 2DFS-STAR still avoids the stop-and-pull delay. For the video analytics bundle, 2DFS-STAR is 7 times faster than eStargz replacement and 7.5 times faster than Docker or 2DFS replacement. Repeating the measured transfer volume across 100 updates would move 9.6 GiB with 2DFS-STAR, 67.1 GiB with eStargz, and 123.9 GiB with Docker or 2DFS. The smaller refresh payload becomes more consequential when all bytes cross the 1 GbE link.

5.9 Compression Method

2DFS-STAR builder supports gzip and zstd:chunked. We isolate their effect by fixing the workload, allotment layout, split capacity, and accessed fraction. Figure 5.11 compares

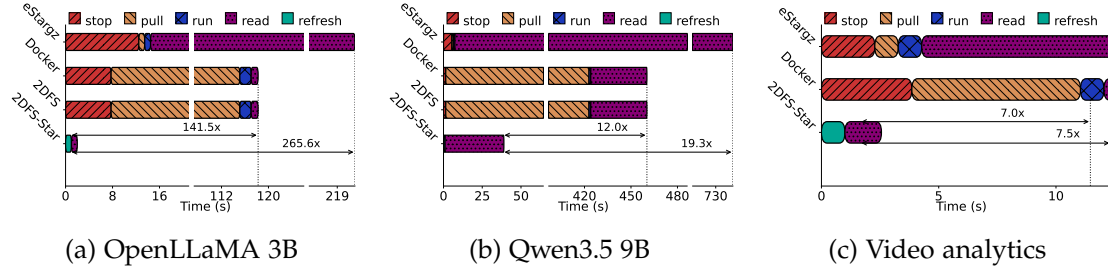


Figure 5.10: Live refresh latency on the edge client.

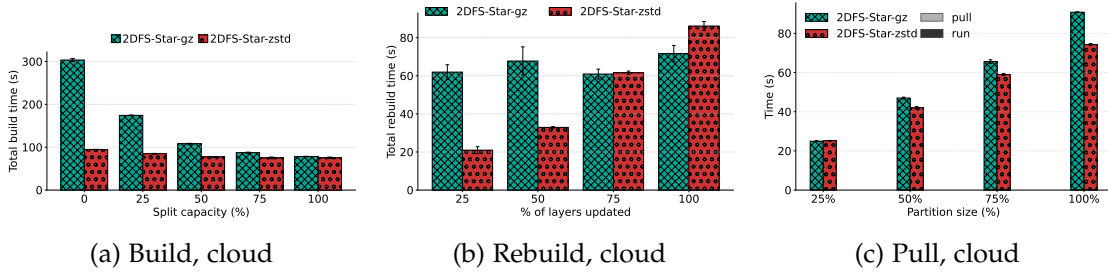


Figure 5.11: Compression method for 2DFS-STAR on Qwen3.5 9B: gzip against zstd:chunked.

build, rebuild, and pull latency for Qwen3.5 9B. Compression method matters most for the unsplit image, where one large unit is compressed serially. The difference narrows when finer partitioning exposes more compression work that can run in parallel.

Both formats preserve on-demand fetching and leave unrequested content remote. Their compression ratios and decompression costs determine the remaining latency difference. zstd:chunked performs better in these experiments. We therefore use it for every other result unless stated otherwise.

5.10 FUSE Passthrough

FUSE passthrough removes the stargz snapshotter daemon from the file read path after a file becomes local (Subsection 4.3.1). We measure its effect on OpenLLaMA 3B in the cloud environment. Figure 5.12 reports pull and run time over three runs at four partition sizes. Passthrough is faster at every size. It reduces latency from 21.3 to 8.9 s at one quarter of the image and from 79.5 to 28.5 s at the full image. The margin widens with the partition because more reads can use local files directly. We therefore enable passthrough in all experiments.

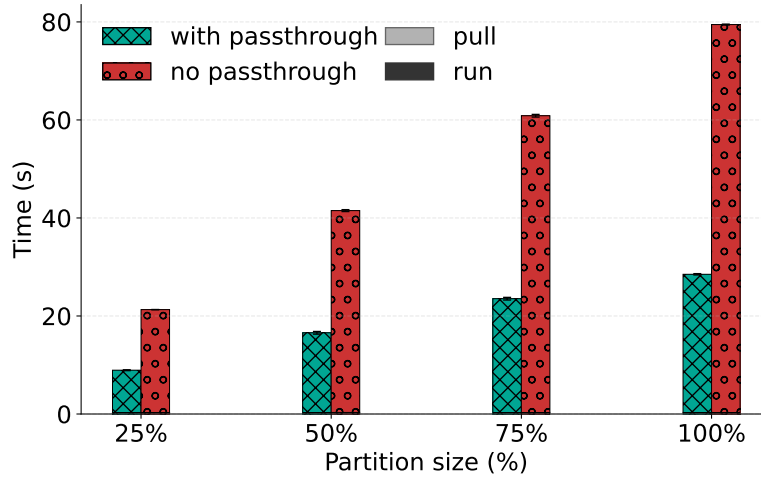


Figure 5.12: Pull and run time for 2DFS-STAR on OpenLLaMA 3B with FUSE passthrough on and off.

5.11 Prefetch

Prefetch downloads a range of every selected layer before the container starts (Subsection 4.3.1). It can hide later read latency when enough time separates the fetch from first access. Our workload reads the model files immediately after the pull. Prefetch and active reads can therefore request the same content concurrently. We compare four snapshotter configurations on Qwen3.5 9B in the cloud environment: prefetch disabled, synchronous prefetch, asynchronous prefetch, and asynchronous prefetch with background fetch. This experiment contains one run per configuration. Figure 5.13 presents the four timelines.

In these runs, disabling prefetch is fastest from 50% partition size upward. Its lead grows with partition size. At full access, the configuration without prefetch finishes in 68s. Synchronous prefetch takes 118s, asynchronous prefetch takes 115s, and background fetch increases the result to 129s. Synchronous prefetch blocks launch for 8.3s at one quarter of the image and for 42.3s at full access. The subsequent run still takes about as long as the configuration without prefetch. Asynchronous prefetch removes this initial wait but competes with active reads. It helps only at the smallest partition, where it takes 21.8s compared with 25.4s without prefetch. These timelines motivate disabling prefetch in all of our measurements.

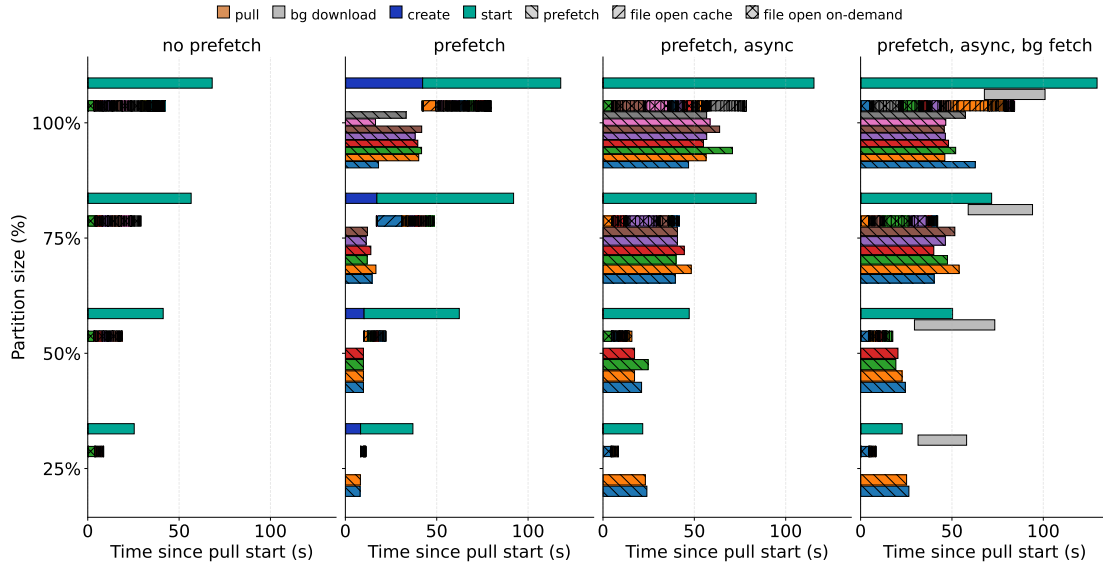


Figure 5.13: Pull, prefetch, file open, and container run timelines for 2DFS-STAR on Qwen3.5 9B under four prefetch configurations, one row per partition size. In the prefetch and file open lanes each color is one layer, so a color shows where the same layer is prefetched and where it is read.

6 Conclusion

We presented 2DFS-STAR, which combines independently addressable image partitions with lazy loading and in-place refresh for large containerized applications.

6.1 Summary of Findings

Our system preserves the granularity of each change throughout the image lifecycle. The measurements in Chapter 5 confirm this at every stage, from the initial build to a live update.

2DFS-STAR builds faster than both Docker and eStargz, and stays close to plain 2DFS, emitting seekable, lazy-loadable allotments in a single pass. The same independent-allotment structure bounds rebuild work: when only part of an image changes, 2DFS-STAR rebuilds just the affected allotments.

Pull and cold-start latency match eStargz and increase only with the fraction of each model that is read. This benefit also holds on the edge client, where other baselines pay the highest cost for downloading and unpacking an image.

Refresh updates in place and transfers only what changed. It reduces update latency by 8.3 to 85.9 times in the cloud and 7.0 to 265.6 times at the edge for the measured workloads, and transfers orders of magnitude less data than manual replacement.

6.2 Applicability Beyond Model Serving

2DFS-STAR applies to any use case that pairs a stable runtime with large artifacts updated independently of the surrounding code. Simulators and game engines, for example, often ship a fixed engine alongside large asset bundles. Other software runtimes similarly pair a stable server with a large, frequently updated body of content.

6.3 Limitations

Staying inside the OCI runtime and image standards caps how finely an image can be partitioned. The OverlayFS lower-layer limit holds an image of up to 128 layers [18], and because each allotment is serialized as its own OCI layer, that ceiling also caps how

many allotments an image can address independently. The number falls further when the base image already uses several layers for the runtime.

Live refresh carries its own preconditions. Within a layer, a file or directory that is added, removed, or renamed cannot be aligned chunk by chunk, so 2DFS-STAR falls back to re-reading the layer lazily. An image whose layer count or order changes cannot be refreshed in place and must instead be rebuilt and pulled again. The refresh diff also compares file contents without considering metadata changes.

Because refresh acts on an image reference, it updates every container running the specified image version. It also offers no consistency guarantee above chunk granularity: a read during refresh may combine updated chunks with stale ones.

Subscription mode has no backoff. After ten failed polls in a row the subscription is dropped. Subscription state is stored in daemon memory and does not survive a restart.

6.4 Future Work

These limitations motivate work on relaxing the structural constraints on refresh, coordinating updates with the applications that consume them, and integrating refresh with checkpointing.

6.4.1 Lifting the Refresh Preconditions

Supporting changes to the number or order of layers requires moving the merged filesystem view from OverlayFS into the daemon. The daemon could then modify the layer set while the container is running. This approach would move every path lookup into userspace, and its performance cost remains to be evaluated.

Metadata-only updates require an invalidation path for file attributes. Keeping containerd's image record consistent after a refresh similarly requires a write-back through its image service, together with appropriate failure handling.

Layer-level locks could make refresh atomic by directing each read entirely to either the old or the new image version. Adding container identity to the chunk cache could also allow a refresh to target one container rather than every container using the same image reference.

Subscription mode could use exponential backoff with jitter and persist subscriptions across daemon restarts. Together, these changes would allow clients to recover from a registry outage without manual intervention. Polling would still impose one HEAD request per subscription and interval, creating increasing registry load as a fleet grows. Since 2DFS-STAR controls the registry, it could replace polling with notifications when

a new tag is pushed. This would reduce both registry load and the delay between publication and refresh.

6.4.2 Application-Aware Coordination

2DFS-STAR cannot coordinate a refresh with the application using affected files. The daemon can make updated bytes available, but it does not know whether the application is currently serving requests from those files or when it is safe to reload them. Conversely, the application cannot observe refresh progress or determine when all updated data is available. Updating the mounted files also does not replace data that the application has already loaded into memory. A model server may therefore continue serving the old weights even after the refresh has completed.

A coordination interface could carry this information in both directions. Before a refresh, a model server could drain requests that use the affected files and signal when it has reached a safe update point. The daemon could then perform the refresh and report its completion to the server. The server could reload the updated model and resume routing requests to it. This protocol would prevent the application from reading a mixture of old and new data during the refresh and ensure that the completed update is subsequently loaded into application memory.

6.4.3 Complementarity with Checkpointing

2DFS-STAR can supply the filesystem side of checkpoint and restore. A checkpoint captures the running state of a process for restart later or on another host [17], and the restored process needs the same files as when the checkpoint was taken. 2DFS-STAR can bind each checkpoint to the digest of the mounted image and the digests of its layers. The stargz snapshotter can record the chunks read up to the checkpoint as the filesystem working set. On restore, the runtime can rebuild the same read-only view, prefetch that working set, and restore process state. As a result, recovery does not require redistributing the full image.

A model server checkpointed this way carries process state only. The checkpoint holds request queues, key-value caches, and session state at a safe point, and the mounted image supplies the weights. Keeping the weights out of the checkpoint is what makes live migration and failure recovery fast.

Abbreviations

CLI	command-line interface
CPU	central processing unit
CRIU	Checkpoint/Restore in Userspace
CV	computer vision
FUSE	Filesystem in Userspace
HTTP	Hypertext Transfer Protocol
ICMP	Internet Control Message Protocol
JSON	JavaScript Object Notation
LLB	Low-Level Build
ML	machine learning
OCI	Open Container Initiative
SATA	Serial ATA
TOC	table of contents

List of Figures

3.1	2DFS-STAR architecture. 2DFS-STAR builder produces an OCI+2DFS image, 2DFS-STAR registry serves it with per-layer prefetch annotations, and the client stack pulls, mounts, and refreshes it. New and modified parts are highlighted relative to base 2DFS and the stargz snapshotter. .	16
3.2	Data distribution in 2DFS-STAR with prefetch and lazy loading. Solid arrows trace the prefetch path for the requested partition; dotted arrows trace an on-demand read of a file outside it.	19
3.3	In-place refresh and subscription in 2DFS-STAR. A refresh is triggered by an explicit command or by a subscription poll; the daemon diffs the changed layers at chunk granularity, swaps the layer readers, and invalidates only the changed cache ranges.	21
5.1	Build time of 2DFS-STAR against Docker, 2DFS, and eStargz across three language models, as a function of split capacity.	33
5.2	Rebuild time against the fraction of modified files, with the top row changing files bottom-to-top and the bottom row top-to-bottom.	34
5.3	Pull plus first-read latency as the application accesses 25% to 100% of each model. The hatched lower segment is the pull, the solid upper segment the read.	35
5.4	Live refresh against manual replacement. The top row is end-to-end update latency; the bottom row is the data transferred during one update.	36
5.5	Resource usage on the builder against split capacity. The top row is CPU time in core-seconds, the bottom row memory held in gigabyte-seconds.	37
5.6	Resource usage on the client during pull plus first read of Qwen3.5 9B, against partition size.	38
5.7	Build and rebuild of the video analytics bundle of four independent CV models.	39
5.8	Pull and refresh of the video analytics bundle.	40
5.9	Pull performance on the resource-constrained edge client.	40
5.10	Live refresh latency on the edge client.	41
5.11	Compression method for 2DFS-STAR on Qwen3.5 9B: gzip against zstd:chunked.	41

5.12	Pull and run time for 2DFS-STAR on OpenLLaMA 3B with FUSE passthrough on and off.	42
5.13	Pull, prefetch, file open, and container run timelines for 2DFS-STAR on Qwen3.5 9B under four prefetch configurations, one row per partition size. In the prefetch and file open lanes each color is one layer, so a color shows where the same layer is prefetched and where it is read.	43

List of Tables

2.1	Capabilities provided by Docker, eStargz, 2DFS, and 2DFS-STAR.	11
4.1	Build-time compression flags on <code>tdfs build</code>	24
5.1	The workloads, the base image each builds on, and the model files it carries. Base image sizes are compressed.	32
5.2	The four models in the video analytics bundle.	32

Bibliography

- [1] N. Asadi, H. İ. Bengu, L. Wulfert, H. Wöhrle, and W. Kellerer. “Road to tiny reality: Digital twins for decentralized ai on microcontrollers.” In: *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. 2025, pp. 1305–1307.
- [2] G. Bartolomeo, N. Asadi, W. Kellerer, J. Ott, and N. Mohan. “{On-Demand} Container Partitioning for Distributed {ML}.” In: *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. 2025, pp. 1481–1500.
- [3] G. Bartolomeo, N. Asadi, W. Kellerer, J. Ott, and N. Mohan. *2DFS: A two-dimensional filesystem build and distribution framework for containers*. <https://2dfs.github.io/>. Accessed: June 7, 2026. 2025.
- [4] G. Bartolomeo, N. Asadi, W. Kellerer, J. Ott, and N. Mohan. “Beyond Layers: Container Registries for Files Distribution and On-Demand Image Partitioning.” In: *20th European Conference on Computer Systems (EuroSys’ 25)-Posters*. ACM. 2025.
- [5] R. Bhardwaj, Z. Xia, G. Ananthanarayanan, J. Jiang, Y. Shu, N. Karianakis, K. Hsieh, P. Bahl, and I. Stoica. “Ekya: Continuous learning of video analytics models on edge compute servers.” In: *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 2022, pp. 119–135.
- [6] CARLA Simulator Team. *CARLA in Docker*. https://carla.readthedocs.io/en/latest/build_docker/. CARLA Documentation, accessed 2026-06-11. 2026.
- [7] D. Chait-Roth, K. S. Namjoshi, and T. Wies. “Consistent Updates for Scalable Microservices.” In: *Proceedings of the ACM on Programming Languages* 10.POPL (2026), pp. 1673–1701.
- [8] J. L. Chen, D. Liaqat, M. Gabel, and E. de Lara. “Starlight: Fast container provisioning on the edge and over the {WAN}.” In: *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 2022, pp. 35–50.
- [9] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam. “Rethinking atrous convolution for semantic image segmentation.” In: *arXiv preprint arXiv:1706.05587* (2017).

- [10] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam. “Encoder-decoder with atrous separable convolution for semantic image segmentation.” In: *Proceedings of the European conference on computer vision (ECCV)*. 2018, pp. 801–818.
- [11] Cloud Native Computing Foundation. *What 500+ Experts Revealed About Kubernetes Adoption and Workloads*. Accessed 2026-06-07. Aug. 2025.
- [12] Y. Collet and M. Kucherawy. *Zstandard Compression and the ‘application/zstd’ Media Type*. RFC 8878. Feb. 2021. DOI: 10.17487/RFC8878.
- [13] CommandLinux. *Linux Server Market Share*. Accessed 2026-06-07. Dec. 2025.
- [14] containerd Authors. *Snapshotters*. <https://github.com/containerd/containerd/blob/main/docs/snapshotters/README.md>. containerd Documentation, accessed 2026-08-13. 2026.
- [15] containerd Authors. *Stargz Snapshotter: Fast Container Image Distribution Plugin with Lazy Pulling (eStargz)*. <https://github.com/containerd/stargz-snapshotter>. Accessed 2026-06-08.
- [16] M. Copik, G. Kwasniewski, M. Besta, M. Podstawski, and T. Hoefler. “SeBS: A Serverless Benchmark Suite for Function-as-a-Service Computing.” In: *Proceedings of the 22nd International Middleware Conference*. Middleware ’21. Québec city, Canada: Association for Computing Machinery, 2021, pp. 64–78. ISBN: 9781450385343. DOI: 10.1145/3464298.3476133.
- [17] CRIU Project. *CRIU: Checkpoint/Restore in Userspace*. <https://criu.org>. Accessed 2026-06-08.
- [18] Docker Inc. *OverlayFS Storage Driver*. <https://docs.docker.com/engine/storage/drivers/overlayfs-driver/>. Docker Documentation, accessed 2026-06-11. 2026.
- [19] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun. “CARLA: An Open Urban Driving Simulator.” In: *Conference on Robot Learning (CoRL)* (2017).
- [20] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, E. Eide, L. Stoller, M. Hibler, D. Johnson, K. Webb, A. Akella, K. Wang, G. Ricart, L. Landweber, C. Elliott, M. Zink, E. Cecchet, S. Kar, and P. Mishra. “The Design and Operation of CloudLab.” In: *Proceedings of the USENIX Annual Technical Conference (ATC)*. July 2019, pp. 1–14.
- [21] Y. Fu, L. Xue, Y. Huang, A.-O. Brabete, D. Ustiugov, Y. Patel, and L. Mai. “{ServerlessLLM}:{Low-Latency} serverless inference for large language models.” In: *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 2024, pp. 135–153.

- [22] gem5 Project Contributors. *gem5 Development Container Image*. <https://github.com/gem5/gem5/pkgs/container/devcontainer>. GitHub Container Registry package, accessed 2026-06-11. 2026.
- [23] Google. *Android Emulator Container Scripts*. <https://github.com/google/android-emulator-container-scripts>. GitHub repository, accessed 2026-06-11. 2026.
- [24] Google DeepMind. *Gemma 4 31B*. <https://huggingface.co/google/gemma-4-31B>. Hugging Face model card, accessed 2026-06-11. 2026.
- [25] Google DeepMind. *Gemma 4 Model Overview*. <https://ai.google.dev/gemma/docs/core>. Google AI for Developers documentation, accessed 2026-06-11. 2025.
- [26] Google LLC. *CRFS: Container Registry Filesystem*. <https://github.com/google/crfs>. Accessed: June 7, 2026. 2019.
- [27] T. Harter, B. Salmon, R. Liu, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. "Slacker: Fast distribution with lazy docker containers." In: *14th USENIX Conference on File and Storage Technologies (FAST 16)*. 2016, pp. 181–195.
- [28] C. M. Hayden, E. K. Smith, M. Denchev, M. Hicks, and J. S. Foster. "Kitsune: Efficient, general-purpose dynamic software updating for C." In: *Proceedings of the ACM international conference on Object oriented programming systems languages and applications*. 2012, pp. 249–264.
- [29] K. He, X. Zhang, S. Ren, and J. Sun. "Deep residual learning for image recognition." In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [30] Jekyll. *Jekyll: Simple, blog-aware, static sites*. <https://jekyllrb.com>. Accessed 2026-06-09.
- [31] M. Khani, G. Ananthanarayanan, K. Hsieh, J. Jiang, R. Netravali, Y. Shu, M. Alizadeh, and V. Bahl. "{RECL}: Responsive {Resource-Efficient} continuous learning for video analytics." In: *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 2023, pp. 917–932.
- [32] X. Z. Khooi, A. Kalia, and M. C. Chan. "How to Update Your 5G vRAN." In: *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. 2025, pp. 123–138.
- [33] S. Lee and O. Contributors. *Open5GS*. <https://open5gs.org>. Accessed: 2026-06-11. 2026.
- [34] H. Li, Y. Yuan, R. Du, K. Ma, L. Liu, and W. Hsu. "{DADI}:{Block-Level} Image Service for Agile and Elastic Application Deployment." In: *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. 2020, pp. 727–740.

- [35] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, et al. “{AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving.” In: *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 2023, pp. 663–679.
- [36] Linux Kernel Contributors. *FUSE Overview*. <https://www.kernel.org/doc/html/latest/filesystems/fuse/fuse.html>. Linux Kernel Documentation, accessed 2026-08-13. 2026.
- [37] C. Lou, S. Qi, C. Jin, D. Nie, H. Yang, Y. Ding, X. Liu, and X. Jin. “{HydraServe}: Minimizing Cold Start Latency for Serverless {LLM} Serving in Public Clouds.” In: *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 2026, pp. 415–430.
- [38] MLflow Contributors. *Serving Multiple Models on a Single Endpoint with a Custom PyFunc*. <https://mlflow.org/docs/latest/ml/traditional-ml/tutorials/serving-multiple-models-with-pyfunc/>. MLflow Documentation, accessed 2026-06-11. 2026.
- [39] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, et al. “Ray: A distributed framework for emerging {AI} applications.” In: *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 2018, pp. 561–577.
- [40] S. Oh and J. Kim. “Stateful container migration employing checkpoint-based restoration for orchestrated container clusters.” In: *2018 International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE. 2018, pp. 25–30.
- [41] Ollama. *Ollama Docker Image*. <https://hub.docker.com/r/ollama/ollama>. Docker Hub repository, accessed 2026-08-09.
- [42] Open Container Initiative. *OCI Image Format Specification*. <https://github.com/opencontainers/image-spec>. Accessed 2026-06-08.
- [43] Open5GS contributors. *Open5GS*. <https://github.com/open5gs/open5gs>. Open-source 5G Core and EPC implementation. 2026.
- [44] OpenLM Research. *open_llama_3b*. https://huggingface.co/openlm-research/open_llama_3b. Hugging Face model card, accessed 2026-06-11. 2023.
- [45] Qwen Team. *Qwen3.5-9B*. <https://huggingface.co/Qwen/Qwen3.5-9B>. Hugging Face model card, accessed 2026-06-11. 2026.
- [46] Ray Project Contributors. *Deploy Compositions of Models*. https://docs.ray.io/en/latest/serve/model_composition.html. Ray Serve Documentation, accessed 2026-06-11. 2026.

- [47] J. Redmon and A. Farhadi. “YOLO9000: better, faster, stronger.” In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 7263–7271.
- [48] J. Redmon and A. Farhadi. “Yolov3: An incremental improvement.” In: *arXiv preprint arXiv:1804.02767* (2018).
- [49] A. Rehn and U. C. C. Contributors. *Microservices*. <https://unrealcontainers.com/docs/use-cases/microservices>. Unreal Containers Documentation, accessed 2026-06-11. 2023.
- [50] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini. “Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider.” In: *2020 USENIX annual technical conference (USENIX ATC 20)*. 2020, pp. 205–218.
- [51] S. S. Shubha and H. Shen. “AdaInf: Data Drift Adaptive Scheduling for Accurate and SLO-guaranteed Multiple-Model Inference Serving at Edge Servers.” In: *Proceedings of the ACM SIGCOMM 2023 Conference*. 2023, pp. 473–485.
- [52] S. S. Shubha, H. Shen, and G. Ananthanarayanan. “CIS: Checkpointed Inference for Data Drift-Resilient Model Serving at Edge Servers.” In: *Proceedings of the 2025 ACM Symposium on Cloud Computing*. 2025, pp. 600–613.
- [53] P. Souza, T. Ferreto, and R. Calheiros. “Maintenance operations on cloud, edge, and IoT environments: Taxonomy, survey, and research challenges.” In: *ACM Computing Surveys* 56.10 (2024), pp. 1–38.
- [54] M. Tan and Q. Le. “Efficientnetv2: Smaller models and faster training.” In: *International conference on machine learning*. PMLR. 2021, pp. 10096–10106.
- [55] G. Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, et al. “Gemma: Open models based on gemini research and technology.” In: *arXiv preprint arXiv:2403.08295* (2024).
- [56] TensorFlow Authors. *TensorFlow Docker Images*. <https://hub.docker.com/r/tensorflow/tensorflow>. Docker Hub repository, accessed 2026-08-09.
- [57] Uvicorn. *Uvicorn: An ASGI web server implementation for Python*. <https://uvicorn.dev>. Accessed 2026-06-09.
- [58] I. Verma and R. Iontel. *Building an Adaptable 5G Core on an Open Source Container Platform*. <https://www.redhat.com/en/blog/building-adaptable-5g-core-open-source-container-platform>. Red Hat Blog, accessed 2026-06-11. Aug. 2021.
- [59] vLLM Project. *Easy, fast, and cheap LLM serving for everyone*. <https://github.com/vllm-project/vllm>. Accessed 2026-06-09.

- [60] B. Wan, M. Han, Y. Sheng, Y. Peng, H. Lin, M. Zhang, Z. Lai, M. Yu, J. Zhang, Z. Song, et al. “{ByteCheckpoint}: A unified checkpointing system for large foundation model development.” In: *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 2025, pp. 559–578.
- [61] A. Wang, S. Chang, H. Tian, H. Wang, H. Yang, H. Li, R. Du, and Y. Cheng. “{FaaSNet}: Scalable and fast provisioning of custom serverless container runtimes at alibaba cloud function compute.” In: *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 2021, pp. 443–457.
- [62] Q. Weng, W. Xiao, Y. Yu, W. Wang, C. Wang, J. He, Y. Li, L. Zhang, W. Lin, and Y. Ding. “{MLaaS} in the wild: Workload analysis and scheduling in {Large-Scale} heterogeneous {GPU} clusters.” In: *19th USENIX symposium on networked systems design and implementation (NSDI 22)*. 2022, pp. 945–960.
- [63] B. Wu, Z. Zhang, Z. Bai, X. Liu, and X. Jin. “Transparent {GPU} sharing in container clouds for deep learning workloads.” In: *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 2023, pp. 69–85.
- [64] Y. Xiang, X. Li, K. Qian, Y. Zhang, W. Yu, E. Zhai, X. Jin, and J. Zhou. “{ServeGen}: Workload Characterization and Generation of Large Language Model Serving in Production.” In: *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 2026, pp. 1845–1859.
- [65] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. “Qwen3 technical report.” In: *arXiv preprint arXiv:2505.09388* (2025).
- [66] H. Zhang, M. Alhanahnah, F. A. Ahmed, D. Fatih, P. Leitner, and A. Ali-Eldin. “Machine learning systems are bloated and vulnerable.” In: *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 8.1 (2024), pp. 1–30.